



Produkt Rapport

Big Dawgs – Tick basseret browser RTS

Lavet af:

Daniel Steenberg

Kevin Nielsen

Kim Mortensen

Afleveringsdato:

D.20-05-2026



Indledning	5
Mål	5
Funktionelle krav	5
Ikke Funktionelle krav	5
Afgrænsninger	5
Produktdokumentation:	
Kort beskrivelse af opbygning:	6
Gameplay og brugeroplevelse (Kim - Ruby on rails):	6
Big Dogs: Rise of the Pack – idéen bag spillet	6
Tema	7
Forside og login-system	7
Navbar og resource bar	8
Pack	8
Buildings	9
Marketplace	9
Map	9
Profil	9
Buildings	10
Dog Bone Factory	10
Dog Coin Den	10
The Doghouse	10
The Dog Kennel	10
Market	11
Marketplace	11
Map	12
Gameplay loop	12
Teknisk opbygning af Rails-applikationen (Kim - Ruby on rails):	13
Hvad er Ruby on Rails	13
Hvorfor ruby on rails	13
Active Record og databaser	14
Controllere	15
Views	16

Server-rendering i Ruby on Rails	17
Routes og params	19
Sikkerhed	19
SQL injection-beskyttelse	19
CSRF-beskyttelse	21
Strong Parameters	21
Sessions og cookies	23
Password hashing	24
Vores valg af sikkerhed	25
Hotwire	26
Turbo	27
Stimulus	29
Gems	32
Projektets arkitektur og opbygning	35
Services	35
Faraday Service	36
JsonWebToken Service	38
MarketBot Service	38
ResourcesFetchingService	39
ViewModels og Domain Models	40
Buildings ViewModel	41
Domain Models	42
Rendering i views	44
Solid Queue	45
Hvordan Solid Queue fungerer	45
Recurring jobs	47
Engine(Backend - Daniel - Go):	49
Struktur:	49
Opstart	49
DB	52
Modeller:	54
Ruter	62
Handlers	64
Tick systemmet	67
CI/CD(DevOps - Daniel - Dokploy):	68
API(Backend - Kevin - C#):	70
Introduktion	70

Overordnet struktur	70
API-design og REST-principper	71
Service-lag og forretningslogik	71
Inputvalidering og robusthed	72
Supply og demand model	72
Performance og memory-overvejelser	73
Singleton design og thread-safety	74
Skalering og videreudvikling	74
API og sikkerhed	75
BotService	75
DTO'er og API-design	76
Swagger og udviklingsværktøjer	76
Testing strategi	77
Refleksion over testing	77
Refleksion over implementeringen	78
Mit bidrag	78
Mangler:	79
Rails	79
Engine:	79
API	80
Konklusion:	81
Bilagsliste:	82
Bilag 1	82
Billag 2	82
Billag 3	83
Billag 4	84
Billag 5	84
Billag 6	85
Billag 7	85
Billag 8	86
billag 9	86
Bilag 10:	87
Bilag 11:	88
Bilag 12:	88
Bilag 13:	89
Bilag 14:	89
Bilag 15:	90
Bilag 16:	90

Kravspecifikation

Indledning

Uptime Daddy er et monitorerings værktøj, det ligger som en online website og bruges til at brugere kan vælge hvilke hjemmeside de gerne vil modtage data omkring, samt at se historisk data. Værktøjet skal kunne modtage websites og på en nem og grafisk måde give brugeren et overblik over statusen på de indtastede websites.

Mål

Målet med projektet har været at udvikle en bruger venlig platform som nemt og hurtigt giver et overblik over statuser på websites, det skal være brugeren der vælger disse websites hvorefter platformen skal levere grafer og stats.

Funktionelle krav

- Spilleren skal kunne oprette en konto og logge ind
- Spilleren skal kunne opbygge og udvikle en koloni
- Spillet skal automatisk producere ressourcer over tid
- Spilleren skal kunne bygge og opgradere bygninger
- Spilleren skal kunne handle på et marked
- Spillet skal understøtte flere samtidige spillere
- Spilleren skal løbende kunne se opdateret spilstatus

Ikke Funktionelle krav

- Systemet skal kunne håndtere samtidige spillere
- Opdateringer skal ske med lav forsinkelse
- Systemet skal være opdelt i services med klare ansvarsområder
- Systemet skal være skalerbart og vedligeholdelsesvenligt
- Kommunikation mellem services skal ske via API'er

Afgrænsninger

- Kun manuelle password reset derved ingen glemt password funktionalitet
- Ingen bruger validering alle fake emails og random navne tilladt

Produktdokumentation:

Kort beskrivelse af opbygning:

Projektet er opbygget som en microservice arkitektur for at benytte specifikke sprog til specifikke problemer og er helt generelt har vi en 3 delt opbygning nemlig: Frontend og backend. Vi vil nedenfor gennemgå hvilke teknologier der er brugt i disse kategorier.

Services:

App (frontend - Kim - Ruby on rails)

Accounts (backend - Kim - Ruby on rails)

API (backend - Kevin - .net)

Engine(Backend - Daniel - Go)

Gameplay og brugeroplevelse (Kim - Ruby on rails):

Big Dogs: Rise of the Pack – idéen bag spillet

Big Dogs: Rise of the Pack er et multiplayer strategispil inspireret af spil som Travian og Grepolis. Formålet med projektet var at udvikle et browserbaseret spil, hvor spilleren gradvist opbygger sin base ved hjælp af ressourcer og bygninger.

Spillets grundidé er, at spilleren bygger og opgraderer forskellige bygninger, som producerer ressourcer over tid. Disse ressourcer kan derefter bruges til at udvide sin base yderligere og låse op for nye muligheder i spillet. På længere sigt var planen, at spilleren skulle kunne opbygge flere baser og konkurrere mod andre spillere om at kontrollere flest byer på kortet.

Spillet var designet omkring lange spilrunder, hvor en server kunne køre i flere måneder ad gangen, før spillet blev nulstillet og en ny runde begyndte.

Derudover var det planen at implementere flere multiplayer-funktioner, blandt andet:

- Træning af soldater og kampsystemer mellem spillere
- Mulighed for at angribe andre spilleres byer og stjæle ressourcer
- Et dynamisk marketplace-system, hvor priser automatisk ændrer sig baseret på udbud og efterspørgsel
- Et clan-system, hvor spillere kan samarbejde i grupper

Nogle af disse funktioner blev kun delvist implementeret eller forblev på idéstadiet grundet projektets tidsramme, kun marketplace-systemet er implementeret.

Tema

Valget af temaet til spillet blev hunde. Gruppen brugte en del tid på at diskutere forskellige idéer og universer til spillet, før der blev truffet en endelig beslutning. Da den overordnede gameplay-idé og spillestilen først var på plads, blev gruppen hurtigt enige om, at et hundetema passede godt til projektet.

En af grundene til valget var, at temaet gjorde spillet mere unikt og gav det en mere humoristisk og afslappet identitet sammenlignet med mange traditionelle RTS- og strategispil, som ofte benytter middelalder-, fantasy- eller krigstemaer. Gruppen syntes, at idéen om et strategispil med hunde som centrale figurer var både sjov og anderledes, samtidig med at det gav mulighed for at skabe et mere kreativt univers.

Temaet passede også godt sammen med spillets ressourcer og bygninger. Ressourcer som dog coins, dogs og bones of meat gjorde det muligt at skabe et sammenhængende gameplay-univers med et tydeligt visuelt og tematisk udtryk. Bygninger som kennels, dog houses og marketplaces understøttede ligeledes hundetemaet og gjorde spillet mere genkendeligt.

Derudover gjorde temaet det lettere at skabe et mere casual og tilgængeligt spil, som ikke virkede for seriøst eller tungt. Gruppen vurderede, at dette kunne gøre spillet mere interessant og underholdende for spilleren.

Forside og login-system

Når brugeren åbner hjemmesiden, bliver vedkommende mødt af spillets login-side (*Se bilag 1*). Her har spilleren mulighed for at logge ind på en eksisterende account for at få adgang til sin pack og fortsætte spillet.

Hvis det er første gang brugeren besøger spillet, kan der oprettes en ny account ved at trykke på knappen *Create Pack*, som fører videre til account-oprettelsen (*Se bilag 2*). Her skal brugeren vælge:

- E-mail
- Password
- Navn til sin pack

Når accounten er oprettet, bliver brugeren sendt tilbage til login-siden, hvor der nu kan logges ind med den valgte e-mail og adgangskode.

Første gang en ny spiller logger ind, bliver spilleren mødt af en side, hvor der skal vælges en placering til sin første by (*Se bilag 3*). Her vises et kort over spillets verden med andre spilleres eksisterende byer placeret rundt omkring på kortet. Spilleren skal herefter vælge et ledigt område og placere sin egen by.

Når byen er placeret, bliver spilleren sendt videre til spillets hovedside (*Se bilag 4*) og kan nu begynde at spille.

Denne løsning blev valgt for at gøre starten af spillet mere interaktiv og give spilleren følelsen af at være en del af en fælles verden med andre aktive spillere allerede fra begyndelsen.

Navbar og resource bar

Øverst på skærmen findes to faste bars, som altid er synlige for spilleren. Den øverste bar er spillets navigationsbar, også kaldet navbaren (*Se bilag 5*). Herfra kan spilleren navigere rundt mellem spillets forskellige sider og funktioner.

Hvis spilleren trykker på hundehovedet i navbaren, bliver man sendt tilbage til spillets forside/root page.

Navbaren indeholder følgende sider:

Pack

Pack-siden var planlagt som et samlet overblik over spillerens pack. Her skulle spilleren blandt andet kunne:

- Se information om sin pack
- Administrere soldater
- Håndtere clans
- Få statistik og andre gameplay-informationer

Denne side blev dog aldrig fuldt implementeret inden projektets afslutning.

Buildings

På buildings-siden kan spilleren se alle sine bygninger samt opgradere dem (*Se afsnittet om Buildings*). Denne side fungerer som en central del af spillets progression, da bygninger producerer ressourcer og låser op for nye muligheder.

Marketplace

Marketplace-siden gør det muligt at købe og sælge dog bones for coins (*Se afsnittet om Marketplace*). Systemet er designet omkring dynamiske priser, som ændrer sig baseret på udbud og efterspørgsel.

Map

På map-siden kan spilleren se sin placering på verdenskortet sammen med andre spilleres placeringer (*Se afsnittet om Map*). Kortet er med til at skabe følelsen af en delt multiplayer-verden.

Profil

Profil-siden var planlagt som et sted, hvor spilleren kunne administrere sin brugerprofil, eksempelvis:

- Ændre password
- Se brugerinformationer
- Administrere account-indstillinger

På nuværende tidspunkt er den eneste implementerede funktion muligheden for at logge ud.

Under navbaren findes resource baren (*Se bilag 6*). Her kan spilleren altid se sine nuværende ressourcer samt det maksimale resource cap. Resource baren er synlig på alle sider i spillet, så spilleren konstant har overblik over sine ressourcer under gameplay.

Buildings

Buildings-siden er stedet, hvor spilleren kan bygge og opgradere sine bygninger (se bilag 7). Bygningerne er en central del af spillets progression, da de producerer ressourcer og låser op for nye funktioner i spillet.

Der var planer om at implementere mange forskellige bygningstyper, men inden projektets afslutning blev der udviklet fem hovedbygninger.

Dog Bone Factory

Dog Bone Factory producerer dog bones, som fungerer som en af spillets vigtigste ressourcer. Ressourcerne bruges blandt andet til opgraderinger og til at holde hundene forsynet med mad. Hvis spilleren mangler dog bones, vil det påvirke udviklingen af byen og gøre det vanskeligere at fortsætte progressionen.

Dog Coin Den

Dog Coin Den producerer dog coins, som fungerer som spillets valuta. Coins bruges til:

- Opgradering af bygninger
- Handel på marketplace

Bygningen er derfor vigtig for spillerens økonomi og mulighed for at udvikle sin base hurtigere.

The Doghouse

The Doghouse fungerer som byens hovedkvarter og lagerbygning. Her opbevares spillerens ressourcer, og bygningens level bestemmer samtidig spillerens resource cap.

Derudover fungerer bygningen som en progression-begrænsning, da andre bygninger ikke kan opgraderes til et højere level end spillerens Doghouse. Dette blev designet for at skabe en mere balanceret udvikling af byen.

The Dog Kennel

The Dog Kennel producerer dogs, som er en af spillets ressourcer. Ideen bag bygningen er, at jo større kennelen bliver, desto flere dogs kan produceres og opbevares.

Mathias, det er her hundende knepper for at få hvalpe

Market

Market-bygningen giver spilleren adgang til marketplace-systemet, hvor der kan købes og sælges dog bones.

Uden denne bygning kan spilleren ikke benytte marketplace-funktionen. Det var oprindeligt planen, at opgraderinger af Market skulle give forskellige fordele, eksempelvis bedre handelspriser eller reducerede gebyrer, men disse funktioner blev ikke færdigudviklet inden projektets afslutning.

Marketplace

Marketplace er stedet, hvor spilleren kan købe og sælge dog bones (Se *bilag 8*). Systemet fungerer som spillets handelsplatform og er designet omkring et simpelt supply-and-demand-system.

Marketplace har teoretisk uendelige ressourcer til rådighed, men prisen på dog bones ændrer sig dynamisk afhængigt af aktiviteten på markedet.

Hvis mange spillere sælger store mængder dog bones, bliver markedet oversvømmet med ressourcer, hvilket får prisen til at falde. Omvendt vil prisen stige, hvis mange spillere køber dog bones, da markedet dermed får færre ressourcer til rådighed.

For at skabe mere aktivitet og dynamik på markedet blev der også implementeret bots, som automatisk køber og sælger ressourcer. Formålet med disse bots var at få marketplace til at føles mere levende, selv når der ikke var mange aktive spillere online. Bot-aktiviteten hjælper samtidig med at få priserne til at ændre sig løbende, så markedet ikke står stille.

Formålet med marketplace-systemet var at skabe en mere levende økonomi, hvor både spillere og automatiske handler påvirker markedet og gør handel mere strategisk.

På marketplace-siden kan spilleren derudover se en historik over de seneste handler på markedet. Her vises blandt andet:

- Tidspunkt
- Spiller
- Handelstype
- Værdier og mængder

Dette giver spilleren mulighed for at følge udviklingen på markedet og vurdere, hvornår det bedst kan betale sig at købe eller sælge ressourcer.

Map

På map-siden kan spilleren se sin placering på verdenskortet sammen med andre spilleres byer (*Se bilag 9*).

Planen med map-systemet var, at det skulle fungere som en central del af multiplayer-delen af spillet. Herfra skulle spillere blandt andet kunne:

- Angribe andre spillere
- Udspionere fjendtlige byer
- Udvide deres territorium
- Interagere med andre spillere på kortet

Disse funktioner nåede dog desværre ikke at blive implementeret inden projektets afslutning.

På nuværende tidspunkt kan spilleren:

- Se andre spilleres placeringer
- Se niveauet på andre spilleres main building/Doghouse
- Få et overblik over den fælles verden

Kortet er opbygget som et simpelt grid-system med et fast antal pladser, hvor hver spiller kan placere sin by. Det var oprindeligt planen at udvikle et mere avanceret og visuelt detaljeret kortsystem, men på grund af projektets tidsramme blev der prioriteret at få den grundlæggende funktionalitet implementeret først.

Selvom mapsystemet ikke blev fuldt færdigudviklet, fungerer det stadig som en vigtig del af spillets multiplayer-følelse, da spilleren kan se, at der eksisterer andre aktive spillere i verdenen.

Gameplay loop

Gameplay-loopet i spillet er bygget op omkring produktion og administration af ressourcer. Spilleren har adgang til tre forskellige ressourcer, som løbende bliver produceret af byens

bygninger. Formålet er at producere så mange ressourcer som muligt for at kunne opgradere sine bygninger og dermed udvikle sin by hurtigere.

Spillets progression er primært fokuseret omkring opgraderingen af spillerens main building, The Doghouse, da denne både øger resource cap og fungerer som en begrænsning for, hvor højt andre bygninger kan opgraderes.

Ressourcer bliver genereret pr. tick, hvor ét tick svarer til 15 sekunder. Ved hvert tick beregnes spillerens samlede produktion baseret på bygningernes niveauer.

Derudover giver marketplace-systemet mulighed for en mere strategisk spillestil. Spillere kan forsøge at tjene ekstra coins ved at købe og sælge ressourcer på de rigtige tidspunkter. Hvis en spiller er god til at følge markedets udvikling, kan det være muligt at opnå store fordele gennem handel og udnytte andre spilleres dårlige handler eller ændringer i markedets priser.

Teknisk opbygning af Rails-applikationen (Kim - Ruby on rails):

Hvad er Ruby on Rails

Ruby on Rails er et webframework baseret på programmeringssproget Ruby. Frameworket består af mange forskellige komponenter og værktøjer, som gør det muligt hurtigt at udvikle komplette webapplikationer.

Ruby on Rails følger MVC-princippet (*Model-View-Controller*), som opdeler applikationen i forskellige ansvarsområder:

- **Models** håndterer data og databasen
- **Views** viser data til brugeren
- **Controllers** håndterer logikken mellem bruger og system

Denne struktur gør applikationen mere overskuelig og lettere at vedligeholde.

gjorde det nemmere hurtigt at komme i gang med projektet.

Hvorfor ruby on rails

Rails er samtidig et framework, der er hurtigt at sætte op og udvikle i. Da vi havde valgt en stor og forholdsvis kompleks opgave, var vi hurtigt enige om at vælge en teknologi, vi allerede havde erfaring med, så vi kunne bruge mere tid på selve projektets funktionalitet i stedet for at lære et helt nyt framework fra bunden.

En anden fordel ved Rails er, at meget funktionalitet allerede er bygget ind i frameworket. Det gør udviklingen hurtigere og giver mulighed for hurtigt at lave både frontend, backend og databasehåndtering i samme system.

Active Record og databaser

Ruby on Rails indeholder en ORM (*Object-Relational Mapping*) kaldet Active Record. Active Record gør det muligt at arbejde med databasen direkte gennem Ruby-kode uden selv at skulle skrive SQL-forespørgsler.

Hvis man eksempelvis har en User-model, kan man oprette og hente brugere direkte gennem Ruby:

```
User.create(name: "Kim")
User.find(1)
User.where(name: "Kim")
```

Her:

- Oprettes en bruger med navnet "Kim"
- Findes brugeren med ID 1
- Søgtes efter alle brugere med navnet "Kim"

Hver tabel i databasen har typisk en tilhørende model i Ruby, som repræsenterer dataene og deres logik.

I modellerne kan der opsættes relationer og valideringer, eksempelvis:

```
class User < ApplicationRecord
  has_secure_password
  has_many :settlements, dependent: :destroy
end
```

```
validates :email, presence: true, uniqueness: true
validates :name, presence: true
validates :password, length: { minimum: 6 }, if: -> { password.present? }
validates :password_confirmation, presence: true, if: -> {
password.present? }
  validates :name, length: { maximum: 50 }, allow_blank: false, uniqueness:
true
end
```

Her sikres det blandt andet:

- At brugeren har en unik e-mail
- At passwordet opfylder bestemte krav
- At en bruger kan have mange settlements

Relationerne gør det nemt at arbejde med tilknyttede data i koden. Eksempelvis:

```
kim = User.find(1)
kim.settlements
```

Dette returnerer alle settlements, som er tilknyttet brugeren med ID 1.

Controllers

Controllers håndterer størstedelen af applikationens logik og fungerer som forbindelsen mellem views og modeller. Hver gang en bruger åbner en side eller kalder et endpoint i Rails, rammer requesten en controller.

Alle controllers nedarver fra en fælles controller kaldet ApplicationController. Her kan man placere logik, som skal gælde for hele applikationen.

Eksempel fra projektet:

```
helper_method :current_user
before_action :require_login
```

```
def current_user
  @current_user ||= User.find_by(id: session[:user_id])
end

def require_login
  if current_user
    if current_user.settlements.empty?
      redirect_to new_map_path
    end
  else
    redirect_to session_path
  end
end
```

I dette eksempel bruges `before_action :require_login`, hvilket betyder, at metoden `require_login` bliver kørt automatisk, før en controller handling udføres.

Metoden kontrollerer:

- Om brugeren er logget ind
- Om brugeren har oprettet en settlement

Hvis brugeren ikke er logget ind, bliver vedkommende sendt til login-siden. Hvis brugeren er logget ind, men endnu ikke har placeret sin første by, bliver spilleren sendt til map-siden for at vælge en placering.

Hvis begge krav er opfyldt, får brugeren adgang til den ønskede side.

Views

I Ruby on Rails har hver route typisk et tilknyttet view, som bruges til at vise data til brugeren. Som standard benytter Rails `.erb`-filer (*Embedded Ruby*). En ERB-fil er en almindelig HTML-fil, hvor det er muligt at skrive Ruby-kode direkte i viewet.

Når siden bliver renderet, bliver Ruby-koden evalueret på serveren, hvorefter der genereres almindelig HTML, som sendes til browseren.

I projektet blev det dog valgt at bruge templating-sproget Slim i stedet for standard ERB (*Se mere under Gems*).

Slim blev valgt, fordi syntaksen er mere kompakt og overskuelig, hvilket gør det muligt at skrive

mindre kode og samtidig bevare samme funktionalitet som i ERB-filer.

Et eksempel på Slim-kode:

```
.d-flex
.d-flex.col-12
- if @user.exists?
  h4 = @user.name
```

I Slim betyder:

- . at der oprettes en HTML-class
- Indrykning bestemmer HTML-strukturen
- - betyder, at linjen skal evalueres som Ruby-kode
- = betyder, at returværdien skal vises i HTML-outputtet

Variablen @user kommer fra den controller, som er tilknyttet viewet, eksempelvis:

```
def show
  @user = User.find(1)
end
```

Den samme kode skrevet i en traditionel ERB-fil ville se således ud:

```
<div class="d-flex">
  <div class="d-flex col-12">
    <% if @user.exists? %>
      <h4><%= @user.name %></h4>
    <% end %>
  </div>
</div>
```

Gruppen oplevede, at ERB hurtigt kunne blive uoverskueligt i større views, især når der var meget HTML blandet sammen med Ruby-kode. Slim gjorde derfor views mere læsbare og hurtigere at arbejde med under udviklingen.

Server-rendering i Ruby on Rails

Ruby on Rails fungerer anderledes end mange moderne frontend/backend-frameworks og er bygget med fokus på server-rendering fremfor store mængder JavaScript i frontend.

I et klassisk moderne setup vil man ofte have:

- En frontend skrevet i eksempelvis React eller Vue
- En backend som kun fungerer som API
- Kommunikation mellem frontend og backend gennem JSON-data

Frontendens JavaScript kalder typisk en API, som returnerer data i JSON-format. Frontenden bruger derefter disse data til dynamisk at opdatere siden i browseren. Det samme gælder den modsatte vej, hvor frontend sender JSON-data til backend gennem POST-requests.

Ruby on Rails fungerer anderledes. Her håndterer backend både logik og rendering af HTML-siderne.

Eksempelvis kan en spiller besøge:

<https://hjemmesiden.dk/buildings>

Denne request rammer en BuildingsController og typisk en index-metode. Her udføres controller-logikken først, hvorefter Rails renderer det tilknyttede Slim-view. Når renderingen er færdig, sendes færdig HTML og CSS direkte tilbage til browseren i stedet for JSON-data.

Det betyder, at en stor del af applikationens logik bliver kørt på serveren fremfor i browseren.

Denne tilgang har både fordele og ulemper.

Fordele:

- Mindre JavaScript i frontend
- Hurtigere udvikling
- Lettere strukturering af applikationen
- Mere logik kan skjules på serveren
- Kan være en sikkerhedsmæssig fordel

Ulemper:

- Mindre fleksibilitet i frontend
- Kan være vanskeligere at lave moderne SPA-løsninger (*Single Page Applications*)
- Flere page reloads uden ekstra værktøjer

For at løse nogle af disse udfordringer benytter Rails moderne værktøjer som Hotwire, Turbo og Stimulus (*Se afsnittet om Hotwire*).

Routes og params

I Ruby on Rails er routes koblet direkte sammen med controllers og deres metoder.

Eksempelvis:

- /buildings vil typisk ramme index-metoden i BuildingsController
- /buildings/1 vil typisk ramme show-metoden

Alle controller-metoder har adgang til en indbygget variabel kaldet params, som indeholder data sendt med i requesten.

Eksempel:

```
def show
  @building = Building.find(params[:id])
end
```

Hvis brugeren besøger:

<https://hjemmesiden.dk/buildings/1>

vil params[:id] indeholde værdien 1.

På denne måde gør Rails det nemt at hente data direkte fra URL'en og bruge dem i controller-logikken.

Sikkerhed

I projektet blev der anvendt flere forskellige former for sikkerhed. En stor fordel ved Ruby on Rails er, at frameworket allerede kommer med mange indbyggede sikkerhedsfunktioner.

SQL injection-beskyttelse

Ruby on Rails beskytter automatisk mod mange former for SQL injection gennem Active Record og prepared statements.

SQL injection opstår typisk, når brugerinput indsættes direkte i SQL-forespørgsler som almindelig tekst. Et usikkert eksempel kunne være:

```
email = params[:email]
sql = "SELECT * FROM users WHERE email = '#{email}'"
```

Hvis en bruger indtaster:

```
' OR 1=1 --
```

vil SQL-forespørgslen blive:

```
SELECT * FROM users WHERE email = '' OR 1=1 --'
```

Da 1=1 altid er sand, vil databasen returnere alle brugere i tabellen.

I Ruby on Rails benyttes i stedet Active Record:

```
User.find_by(email: params[:email])
```

Her genererer Rails automatisk en prepared statement:

```
SELECT * FROM users WHERE email = ?
```

Brugerens input bliver herefter sendt som en værdi til databasen i stedet for som en del af selve SQL-koden. Hvis brugeren forsøger at skrive:

```
' OR 1=1 --
```

vil databasen behandle det som almindelig tekst til email-feltet og ikke som SQL-kode. Dermed undgås SQL injection-angrebet.

Denne type sikkerhed er særligt vigtig i Ruby on Rails, da frameworket er bygget op omkring tæt integration mellem frontend, backend og database gennem controllers, views og Active Record.

CSRF-beskyttelse

Ruby on Rails beskytter automatisk mod CSRF-angreb (*Cross-Site Request Forgery*).

Et CSRF-angreb sker, når en bruger er logget ind på en hjemmeside, og en anden hjemmeside forsøger at få brugerens browser til automatisk at sende requests til den side, hvor brugeren allerede er autentificeret.

Eksempelvis kunne en bruger være logget ind på sin bank samtidig med, at vedkommende besøger en ondsindet hjemmeside. Den ondsindede side kunne forsøge at sende en skjult POST-request til banken med eksempelvis en pengeoverførsel.

Problemet opstår, fordi browseren automatisk sender brugerens session-cookie med requesten.

For at beskytte mod dette bruger Rails et såkaldt `authenticity_token`.

Alle forms i Rails får automatisk tilføjet et skjult `authenticity_token`-felt:

```
<input type="hidden" name="authenticity_token" value="...">
```

Når formen bliver sendt, kontrollerer Rails:

- Om tokenet eksisterer
- Om tokenet er gyldigt
- Om tokenet matcher den aktuelle brugers session

Hvis tokenet mangler eller er ugyldigt, afviser Rails requesten.

Det betyder, at en ekstern hjemmeside ikke blot kan sende falske POST-requests til applikationen, da den ikke har adgang til et gyldigt `authenticity_token`.

Denne sikkerhedsmekanisme er indbygget direkte i Rails og aktiveres automatisk på standard forms og controller requests.

Strong Parameters

Ruby on Rails indeholder også en sikkerhedsfunktion kaldet Strong Parameters.

Strong Parameters bruges til at kontrollere, hvilke værdier en controller må modtage fra brugeren. Dette beskytter applikationen mod såkaldt mass assignment, hvor en bruger forsøger at sende ekstra værdier, som ikke burde kunne ændres.

Når en controller modtager data fra eksempelvis en form eller et request, bliver dataene placeret i variablen `params`.

Et request kunne eksempelvis indeholde:

```
{
  user: {
    email: "test@test.com",
    password: "admin",
    role: "admin"
  }
}
```

I controlleren kan man derefter definere hvilke felter, der er tilladt:

```
def user_params
  params.require(:user).permit(:email, :password)
end
```

Her betyder:

- `require(:user)` at requesten skal indeholde et `user`-objekt ellers får du en fejl
- `permit(:email, :password)` at kun disse felter er tilladt

Hvis applikationen derefter opretter en bruger med:

```
User.create(user_params)
```

vil feltet `role` automatisk blive ignoreret, selvom brugeren forsøgte at sende det med.

Dette er vigtigt, da det forhindrer brugere i at manipulere requests og ændre værdier, som de ikke burde have adgang til at kontrollere, eksempelvis `admin`-rettigheder eller andre følsomme databasefelter.

Sessions og cookies

Ruby on Rails har indbygget håndtering af sessions og cookies, som gør det muligt sikkert at holde styr på, hvilken bruger der er logget ind.

I mange moderne systemer bruges JWT (*JSON Web Tokens*) til autentificering, men i dette projekt blev Rails' indbyggede session-system anvendt.

Når en bruger logger ind, kan Rails eksempelvis gemme brugerens ID i sessionen:

```
session[:user_id] = user.id
```

Rails gemmer herefter session-data i en signed og encrypted cookie i browseren.

Det betyder, at dataene:

- Ikke kan læses direkte af brugeren
- Ikke kan ændres uden at Rails opdager det
- Er beskyttet gennem Rails' `secret_key_base`

Selvom sessionen eksempelvis indeholder brugerens ID, vil værdien i browseren fremstå som krypteret data i stedet for almindelig tekst.

Når brugeren besøger nye sider, sender browseren automatisk session-cookien med til serveren. Rails kan herefter dekryptere og validere sessionen og dermed identificere den aktuelle bruger.

I projektet blev dette blandt andet brugt til at hente den nuværende bruger:

```
@current_user ||= User.find(session[:user_id])
```

Her bruges værdien fra sessionen til at finde den tilhørende bruger i databasen.

En fordel ved Rails' session-system er, at meget af sikkerheden er indbygget direkte i frameworket. Rails håndterer automatisk:

- Signering af cookies
- Kryptering af sessionsdata
- Beskyttelse mod manipulation af cookies

Dette gør det muligt at implementere login-systemer med relativt lidt kode samtidig med, at sikkerheden stadig er høj.

Password hashing

Ruby on Rails bruger ofte gemmen `bcrypt` til sikker håndtering af passwords (*Se mere under Gems*).

Ved at tilføje:

```
class User < ApplicationRecord
  has_secure_password
end
```

til en model aktiveres Rails' indbyggede password-håndtering.

Når en bruger eksempelvis opretter en account med:

```
user.password = "secret123"
```

vil Rails automatisk hashe passwordet før det gemmes i databasen.

Passwordet bliver ikke gemt som almindelig tekst, men i stedet som en hash genereret af `bcrypt`. Under hashingen bruger `bcrypt` også et såkaldt salt, som er tilfældige data der gør hver hash unik.

Det betyder, at selv hvis to brugere vælger det samme password, vil deres hashes stadig være forskellige.

Et hashed password kunne eksempelvis se således ud:

```
$2a$12$wQv9K8zYxR1...
```

Rails kan ikke dekryptere passwordet igen, da hashing er en envejsproces.

Når brugeren logger ind, hasher Rails det indtastede password igen og sammenligner resultatet med den hash, som er gemt i databasen. Hvis de matcher, bliver brugeren logget ind.

Denne metode gør det langt mere sikkert at opbevare bruger-passwords, da de rigtige passwords aldrig gemmes direkte i databasen.

Vores valg af sikkerhed

I projektet blev der valgt flere forskellige sikkerhedsløsninger, da systemet består af flere backends og services, som skal kommunikere sikkert med hinanden.

Til autentificering af brugere i Rails-applikationen blev Rails' indbyggede session-system anvendt. Når en bruger logger ind, bliver brugerens ID gemt i en encrypted session-cookie.

I projektet blev følgende metode brugt i ApplicationController:

```
helper_method :current_user
before_action :require_login

def current_user
  @current_user ||= User.find_by(id: session[:user_id])
end

def require_login
  if current_user
    if current_user.settlements.empty?
      redirect_to new_map_path
    end
  else
    redirect_to session_path
  end
end
```

Her bliver `current_user` hentet fra sessionen hver gang en controller bliver kaldt. Hvis brugeren ikke er logget ind, og der derfor ikke findes en gyldig session, bliver brugeren automatisk sendt tilbage til login-siden.

Til kommunikation mellem Rails-applikationen og de andre backend-services blev der anvendt JWT (*JSON Web Tokens*).

JWT blev ikke brugt som hoved-login-system for brugeren i Rails, men i stedet som autentificering mellem services og API-endpoints.

I User-modellen blev der oprettet en metode til at generere tokens:

```
def encoded_json_web_token
  JsonWebTokenService.encode({
    user_id: id,
    email: email,
    iss: "big_dog_rails",
    exp: 5.minutes.from_now.to_i
  })
end
```

Da `current_user` altid er tilgængelig i Rails-applikationen, kunne systemet nemt generere en ny JWT-token, når Rails skulle kommunikere med eksempelvis:

- Game Engine
- Marketplace Engine
- Andre API-services

Tokenen indeholder blandt andet:

- Brugerens ID
- Brugerens e-mail
- Hvilken service der har genereret tokenen
- Et udløbstidspunkt (*expiration time*)

Ved at generere nye tokens løbende med kort levetid blev sikkerheden forbedret, samtidig med at kommunikationen mellem de forskellige services fungerede hurtigt og stabilt.

Hotwire

Hotwire er en teknologi udviklet til Ruby on Rails for at gøre det lettere at lave mere dynamiske webapplikationer uden at skulle skrive store mængder JavaScript.

Hotwire blev udviklet som et alternativ til moderne SPA-løsninger (*Single Page Applications*) som eksempelvis React eller Vue. Målet er at give mange af de samme funktioner, men stadig bevare Rails' server-renderede tilgang, hvor størstedelen af logikken ligger på backend.

Hotwire består primært af:

- Turbo
- Stimulus

Disse arbejder sammen for at skabe mere dynamiske brugeroplevelser i Rails-applikationer.

Turbo

Turbo er den del af Hotwire, som håndterer dynamisk opdatering af sider og HTML-indhold.

Turbo består egentlig af flere dele, men i dette projekt blev især Turbo Frames anvendt.

Turbo Frames gør det muligt at opdele en side i mindre sektioner, som kan opdateres individuelt uden at reloade hele siden.

Dette blev blandt andet brugt til spillets resource bar.

I projektets layout-fil blev resource baren indsat således:

```
- if current_user && current_user.settlements.any?  
  = turbo_frame_tag "resources_bar",  
    src: resource_bar_show_path,  
    data: { controller: "resource-bar" } do  
    = render "layouts/resources_bar"
```

Her:

- Oprettes et Turbo Frame med ID'et resources_bar
- src angiver hvilken route der skal bruges til at hente indholdet
- Resource baren bliver automatisk indlæst, når siden vises

Når Turbo Frame'et loader, kaldes følgende controller:

```
class ResourceBarController < ApplicationController  
  def show  
    @resources_fetching_service = ResourcesFetchingService.new(current_user)  
    @resources_fetching_service.save_to_cache  
  end  
end
```

Controlleren henter spillerens ressourcer og sender dem videre til viewet.

Viewet returnerer derefter HTML-indholdet til Turbo Frame'et:

```
= turbo_frame_tag "resources_bar" do
  .d-flex.col-12.m-0.p-0.justify-content-center.mt-2
  .d-flex.col-6.justify-content-center
  .d-flex.text-white.me-4
  span 🦴
  span ="dog bones:
#{@resources_fetching_service.dog_bones}/#{@resources_fetching_service.resource
_cap.fetch("dog_bones")}"

  .d-flex.text-white.ms-4.me-4
  span 🪙
  span ="coins:
#{@resources_fetching_service.dog_coins}/#{@resources_fetching_service.resource
_cap.fetch("dog_coins")}"

  .d-flex.text-white.ms-4
  span 🐕
  span ="Dogs:
#{@resources_fetching_service.dogs}/#{@resources_fetching_service.resource_cap.
fetch("dogs")}"
```

Turbo undersøger herefter, om det returnerede HTML-indhold indeholder et Turbo Frame med samme ID. Hvis det gør, bliver kun dette frame opdateret i browseren uden at resten af siden ændres.

Dette gjorde det muligt at opdatere resource baren dynamisk uden at reloade hele siden, hvilket gav en mere moderne og flydende brugeroplevelse.

Turbo Frames blev også brugt flere andre steder i projektet til dynamiske opdateringer og delvise sideændringer.

Stimulus

Stimulus er frontend-delen af Hotwire og bruges til den JavaScript-funktionalitet, som ikke kan undgås i browseren.

I stedet for at skrive store JavaScript-filer, som påvirker hele siden, arbejder Stimulus med små

controllers, som kun bliver aktiveret på specifikke dele af HTML'en.

Dette passer godt sammen med Rails' filosofi om at holde størstedelen af logikken på backend og kun bruge JavaScript, hvor det er nødvendigt.

I Stimulus opretter man såkaldte Stimulus Controllers.

En controller er en JavaScript-klasse, som automatisk bliver instansieret på de HTML-elementer, der refererer til den gennem data-controller.

Et simpelt eksempel fra projektet er controlleren til resource baren:

```
import { Controller } from "@hotwired/stimulus"

// Connects to data-controller="resource-bar"
export default class extends Controller {
  connect() {
    setInterval(() => {
      this.element.reload()
    }, 10000)
  }
}
```

Her bruges den indbyggede connect()-metode, som automatisk bliver kørt, når controlleren bliver tilknyttet et HTML-element.

I resource barens Turbo Frame blev controlleren tilknyttet således:

```
- if current_user && current_user.settlements.any?
  = turbo_frame_tag "resources_bar",
    src: resource_bar_show_path,
    data: { controller: "resource-bar" }
```

Linjen:

```
data: { controller: "resource-bar" }
```

fortæller Stimulus, at dette element skal bruge controlleren resource-bar.

Når siden loader, bliver controllerens connect()-metode automatisk kørt, hvilket i dette tilfælde genindlæser Turbo Frame'et hvert 10. sekund.

Dette gjorde det muligt automatisk at opdatere resource baren uden at reloade hele siden.

Stimulus blev også brugt til mere avancerede frontend-funktioner, blandt andet marketplace-sliden.

Controlleren:

```
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["range", "number", "received", "total"]

  static values = {
    rate: { type: Number, default: 1 }
  }

  connect() {
    this.sync(this.rangeTarget.value)
  }

  syncFromRange() {
    this.sync(this.rangeTarget.value)
  }

  syncFromNumber() {
    this.sync(this.numberTarget.value)
  }

  sync(value) {
    const max = Number(this.rangeTarget.max)
    const amount = Math.min(Math.max(Number(value) || 0, 0), max)
    const received = Math.floor(amount * this.rateValue)

    this.rangeTarget.value = amount
    this.numberTarget.value = amount
    this.receivedTarget.textContent = this.format(received)
    this.totalTarget.textContent = this.format(received)
  }

  format(value) {
    return new Intl.NumberFormat().format(value)
  }
}
```

```
}  
}
```

Her bruges flere af Stimulus' funktioner:

- Targets
- Values
- Actions
- Controller-metoder

Controlleren bliver startet på følgende element:

```
.card-body  
data={  
  controller: "marketplace-slider",  
  "marketplace-slider-rate-value": @price  
}
```

Her bliver:

- controller brugt til at vælge hvilken Stimulus-controller der skal bruges
- marketplace-slider-rate-value brugt til at sende data fra Rails til JavaScript

Stimulus gør det også muligt at definere targets:

```
data: {  
  marketplace_slider_target: "range"  
}
```

Targets fungerer som referencer til bestemte HTML-elementer inde i controlleren.

Actions bruges til at koble events til controller-metoder:

```
action: "input->marketplace-slider#syncFromNumber"
```

Her betyder det:

- Når et input-event sker

- Kald controlleren marketplace-slider
- Kør metoden syncFromNumber

Selvom Stimulus kan virke lidt uoverskueligt i starten, gjorde det frontend-koden langt mere organiseret og gjorde det muligt at lave dynamiske funktioner uden at skulle bygge en fuld SPA-løsning med store JavaScript-frameworks.

Gems

I Ruby og Ruby on Rails kaldes udvidelser og eksterne pakker for gems. Gems fungerer som små biblioteker eller pakker med færdiglavet kode, som kan tilføjes til et projekt for at udvide funktionaliteten.

Ruby on Rails kommer allerede med mange gems som standard, men det er også muligt selv at tilføje flere efter behov.

Alle gems i projektet bliver defineret i en fil kaldet Gemfile. Når man installerer projektets gems, bruges værktøjet Bundler, som følger med Ruby. Installation sker typisk med kommandoen:

```
bundle install
```

Bundler downloader herefter alle de gems og versioner, som projektet afhænger af.

Rails autoloader automatisk installerede gems, hvilket betyder, at man i de fleste tilfælde ikke selv behøver at bruge require for at hente dem ind i projektet.

Et udsnit af projektets Gemfile:

```
source "https://rubygems.org"

gem "rails", "~> 8.0.2"
gem "pg", "~> 1.1"
gem "puma", ">= 5.0"
gem "turbo-rails"
gem "stimulus-rails"
gem "slim-rails"
gem "bootstrap", "~> 5.3"
gem "bootstrap-icons"
gem "bcrypt", "~> 3.1.7"
```

```
gem "jwt"  
gem "dotenv-rails"  
gem "solid_queue"  
gem "faraday", "~> 2.0"
```

Selvom det kan virke mærkeligt at have:

```
gem "rails"
```

i et Rails-projekt, er det nødvendigt, fordi Rails-gemmen automatisk installerer en lang række afhængigheder, som frameworket består af.

Eksempelvis:

```
gem "actionpack", "8.0.2"  
gem "activerecord", "8.0.2"  
gem "actionview", "8.0.2"
```

Disse gems håndterer blandt andet:

- Controllers
- Databaser
- Views
- Routing
- Rendering

Ved blot at installere rails får man automatisk alle disse komponenter med.

pg

pg bruges til kommunikation med PostgreSQL-databasen.

puma

puma er webserveren, som kører Rails-applikationen. Den modtager HTTP-requests og sender HTML eller data tilbage til brugeren.

turbo-rails og stimulus-rails

Disse gems bruges til Hotwire-funktionalitet og gør det muligt at lave dynamiske sider uden store mængder JavaScript (*Se afsnittet om Hotwire*).

slim-rails

Gør det muligt at bruge Slim-templates i stedet for standard ERB-filer.

bootstrap og bootstrap-icons

Bruges til styling og design af brugergrænsefladen.

bcrypt

Bruges til hashing af passwords og sikker håndtering af login-oplysninger.

jwt

Bruges til at generere og validere JSON Web Tokens til kommunikation mellem services.

dotenv-rails

Gør det nemt at arbejde med environment variables gennem .env-filer.

solid_queue

Bruges til background jobs og asynkrone opgaver i Rails.

faraday

Bruges til kommunikation med eksterne API'er og backend-services.

Størstedelen af disse gems følger automatisk med Rails, mens enkelte blev tilføjet specifikt til projektet, blandt andet:

- Faraday
- Slim
- Bootstrap
- JWT

Derudover blev der også brugt flere udviklings-gems til debugging og udvikling.

Eksempel:

```
group :development do
  gem "web-console"
end
```

Her betyder `group :development`, at gemmen kun bliver installeret og brugt i development-environmentet.

`web-console` giver blandt andet adgang til en interaktiv fejl-side i browseren, som gjorde debugging og udvikling lettere under projektet.

Projektets arkitektur og opbygning

I *Big Dogs: Rise of the Pack* følger størstedelen af applikationen den klassiske Rails-struktur med controllers, models og views. Projektet indeholder dog også flere ekstra lag og services, som blev udviklet for at gøre kommunikationen mellem de forskellige lag lettere at arbejde med.

Services

Selvom services ikke er unormale i Ruby on Rails, bliver de ikke altid brugt i mindre projekter. I dette projekt blev services anvendt til at samle logik, som skulle bruges flere steder i applikationen.

En service i Rails er typisk blot en almindelig Ruby-klasse, som placeres i projektets service-mappe. Rails autoloader automatisk disse klasser, så de kan bruges direkte i controllers og andre dele af projektet.

Projektet benytter fire services.

Faraday Service

Faraday-servicen bruges til at kommunikere med de eksterne backends og API'er.

Servicen indeholder blandt andet to class methods:

```
def self.post(url, connection_type: "ENGINE_BASE_URL", token: nil, body:
```

```

nil)
  new(url, connection_type).post_request(token: token, body: body)
end

def self.fetch_data(url, connection_type: "ENGINE_BASE_URL", token: nil)
  response = new(url, connection_type).fetch_data(token: token)

  if response.success?
    response.body
  else
    raise "#{response.status}, error: #{response}"
  end
end
end

```

Den ene metode bruges til POST-requests, mens den anden bruges til at hente data fra API'er.

Begge metoder instansierer først klassen:

```

def initialize(url, connection_type)
  @url = url
  @connection_type = connection_type
end

```

Her modtager servicen:

- Den endpoint der skal kaldes
- Hvilken backend der skal kommunikeres med

Projektet bruger 2 backends:

- Engine backend
- Marketplace backend

Derfor blev connection_type brugt til dynamisk at vælge den korrekte base URL gennem environment variables som:

- ENGINE_BASE_URL
- MARKETPLACE_BASE_URL

Servicen opretter herefter en Faraday-forbindelse:

```
def faraday_connection
  @faraday_connection ||= Faraday.new(url: ENV.fetch(:connection_type,
'localhost:5432')) do |faraday|
    faraday.request :json
    faraday.response :json
    faraday.adapter Faraday.default_adapter
  end
end
```

Her bliver Faraday konfigureret til automatisk at:

- Sende JSON-data
- Modtage JSON-data
- Bruge Faradays standard HTTP-adapter

Når forbindelsen er oprettet, kan der eksempelvis udføres et GET-request:

```
def fetch_data(token: nil)
  response = faraday_connection.get(url) do |request|
    request.headers["Authorization"] = "Bearer #{token}" if token
  end
  response
end
```

Hvis nødvendigt, kan der sendes en JWT-token med i requestens Authorization header.

Formålet med denne service var at undgå at skrive den samme API-kode flere steder i projektet. I stedet for at hver controller selv skulle:

- Oprette HTTP-forbindelser
- Håndtere JSON
- Opsætte headers
- Håndtere JWT-tokens

blev dette samlet ét sted, hvilket gjorde koden lettere at genbruge og vedligeholde.

JsonWebToken Service

Projektet indeholder også en JsonWebTokenService, som blev lavet til at encode og decode JWT-tokens.

```
class JsonWebTokenService
  SECRET = ENV.fetch("JWT_SECRET")

  def self.encode(payload)
    JWT.encode(payload, SECRET, "HS256")
  end

  def self.decode(token)
    JWT.decode(token, SECRET, true, { algorithm: "HS256" })[0]
  end
end
```

Servicen blev primært brugt til at encode brugerinformationer inden API-kald til de andre backends.

I praksis blev decode-funktionen dog aldrig brugt særligt meget i Rails-applikationen, da Rails hovedsageligt kun genererede tokens til de andre services.

Det ville muligvis have været mere optimalt at flytte encode-logikken direkte ind i modellen eller et andet sted i projektet, men dette nåede ikke at blive refaktoreret inden projektets afslutning.

MarketBot Service

Projektet indeholder også en MarketBotService, som bruges sammen med background jobs (*Se afsnittet om Solid Queue*).

```
class MarketBotService
  def self.perform
    user = User.first

    FaradayService.post(
      "/market/bot/trade",
      connection_type: "MARKETPLACE_BASE_URL",
```

```

        token: user.encoded_json_web_token
      ).body
    end
  end
end

```

Services bruges til at kalde marketplace-botten, som automatisk udfører handler på markedet for at skabe mere aktivitet og dynamik.

Denne logik blev placeret i en service i stedet for en controller, da background jobs ikke bør være afhængige af controller-logik.

ResourcesFetchingService

Den sidste service er ResourcesFetchingService.

Denne service bruges til at hente spillerens ressourcer fra engine-backenden. Da disse data bruges flere steder i applikationen, blev en service vurderet som den bedste løsning.

Services instansieres med en bruger:

```

class ResourcesFetchingService
  attr_reader :user

  def initialize(user)
    @user = user
  end
end

```

Når servicen er oprettet, kan den blandt andet hente spillerens resource cap:

```

def resource_cap
  @resource_cap ||= FaradayService.fetch_data(
    "/resources/getCap",
    token: user.encoded_json_web_token
  )
end

```

Den kan også hente specifikke ressourcer såsom dog bones:

```
def dog_bones
  @dog_bones = resource_bag.fetch("dogbones")&.fetch("Amount", 0) || 0
end
```

Selve dataene hentes fra engine-backenden:

```
def resource_data
  @resource_data ||= FaradayService.fetch_data(
    "/resources/getBag",
    token: user.encoded_json_web_token
  )
end
```

Servicen gjorde det muligt at samle resource-logik ét sted i stedet for at gentage de samme API-kald i flere controllers og views.

ViewModels og Domain Models

Projektet benytter også en ViewModel til buildings-siden.

En ViewModel bruges typisk, når et view kræver store mængder data eller mere kompleks strukturering af data, uden at der nødvendigvis findes en direkte database-model til det.

I dette projekt bliver alle bygninger hentet fra engine-backenden gennem API-kald i stedet for direkte fra Rails-databasen. Derfor blev det vurderet, at en ViewModel var den bedste løsning til at organisere og håndtere dataene, før de blev sendt videre til viewet.

I controlleren hentes building-dataene først fra backend-servicen:

```
def index
  buildings = FaradayService.fetch_data(
    "/buildings",
    token: current_user.encoded_json_web_token
  )
end
```

```
)

@buildings_view_model = ViewModels::Buildings.new(buildings)
end
```

Her bliver alle bygninger hentet fra API'et og sendt videre til Buildings ViewModelen.

Buildings ViewModel

Når ViewModelen instansieres, gemmer den building-dataene og opretter herefter de forskellige building-objekter:

```
def initialize(buildings_data)
  @buildings = []
  @building_data = buildings_data

  create_buildings
end
```

Selve oprettelsen af bygninger sker i create_buildings:

```
def create_buildings
  building_data.each do |building_data|
    case building_data.fetch('key')
    when 'the_dog_kennel'
      @kennel = Domain::Kennel.new(building_data)
      buildings << @kennel

    when 'the_doghouse'
      @the_doghouse = Domain::TheDoghouse.new(building_data)
      buildings << @the_doghouse

    when 'meat_factory'
      @dog_bone_factory = Domain::DogBoneFactory.new(building_data)
      buildings << @dog_bone_factory

    when 'dog_coin_den'
      @dog_coin_den = Domain::DogCoinDen.new(building_data)
```

```

        buildings << @dog_coin_den

    when 'market'
        @market = Domain::Market.new(building_data)
        buildings << @market
    end
end
end
end

```

Her gennemgås alle bygninger, hvorefter der oprettes en specifik domain-klasse baseret på bygningens type.

Domain Models

Projektet benytter også såkaldte Domain Models til bygningerne.

Alle bygninger nedarver fra en fælles BaseBuilding-klasse:

```

class BaseBuilding
  attr_reader :level,
              :upgrade_cost,
              :image_path,
              :name,
              :production,
              :description,
              :is_constructing,
              :construction_time_left,
              :bones_cost,
              :dog_coins_cost,
              :dogs_cost

  def initialize(building_data, description, image_path = nil)
    @level = building_data.fetch("level", 0)

    upgrade_cost = building_data.fetch("upgrade_cost")

    @image_path = image_path
    @bones_cost = upgrade_cost.fetch("dog_bones", 0)
    @dog_coins_cost = upgrade_cost.fetch("dog_coins", 0)
  end
end

```

```

@dogs_cost = upgrade_cost.fetch("dogs", 0)

@production = building_data.fetch("production", nil)

@name = building_data.fetch("key", "is_constructing")
               .gsub("_", " ")
               .capitalize

@description = description

@is_constructing = building_data.fetch("is_constructing", false)

if @is_constructing
  @construction_time_left =
    Time.parse(building_data.fetch("completes_at")) - Time.now
else
  @construction_time_left = nil
end
end
end

```

BaseBuilding fungerer i praksis som en fælles klasse, der holder på alle de generelle data, som alle bygninger deler.

De enkelte bygningstyper nedarver herefter fra denne klasse og tilføjer deres egne beskrivelser og funktioner.

Eksempelvis Kennel:

```

module Domain
  class Kennel < BaseBuilding
    attr_reader :building_data

    def initialize(building_data)
      @building_data = building_data

      super(
        building_data,
        <<~DESC,

```

```

"Grow your pack. Expand your power."
The Kennel is where your dogs are trained, housed, and prepared for
expansion.
👉 Increases number of dogs
  DESC
  "kennel.png"
)
end

def production
  "Current production:
#{building_data.fetch("production_per_tick").fetch("dogs")} dogs per tick."
end
end
end

```

Her tilføjer Kennel blandt andet:

- Egen beskrivelse
- Eget billede
- Egen production-logik

Denne struktur gjorde det muligt at holde building-logikken mere organiseret og gjorde det lettere at arbejde med forskellige bygningstyper i views.

Rendering i views

I Slim-viewet bliver bygningerne herefter renderet gennem et loop:

```

- @buildings_view_model.buildings.each do |building|
  = render "building/building_card", building: building

```

Her gennemgås alle building-objekter fra ViewModelen, hvorefter der renderes en partial for hver bygning.

Inde i partialen kan der direkte arbejdes med building-objektets data:

```

.card-header.d-flex.justify-content-center.flex-column.align-items-center

```

```
h5.mb-0 = building.name

span.badge.bg-secondary.mt-2.col-2 =
  building.level > 0 ? "Level #{building.level}" : "Not built"
```

Her bruges blandt andet bygningens navn og level direkte fra domain-klassen.

Denne struktur gjorde buildings-systemet mere overskueligt og lettere at udvide med nye bygningstyper senere i projektet.

Solid Queue

Solid Queue er den gem, som projektet bruger til at håndtere background jobs.

Background jobs bruges til opgaver, som skal køres i baggrunden uden at blokere selve hjemmesiden eller brugerens requests. Dette kan eksempelvis være:

- Bots
- Timers
- Scheduled tasks
- Lange API-kald

Hvordan Solid Queue fungerer

Solid Queue fungerer ved at oprette en række database-tabeller, som holder styr på alle jobs i systemet.

I større projekter er det normalt at bruge en separat database til Solid Queue, men i dette projekt blev det vurderet, at det var tilstrækkeligt at bruge samme database som resten af Rails-applikationen.

En af tabellerne er eksempelvis:

```
create_table :solid_queue_jobs do |t|
  t.string :queue_name, null: false
  t.string :class_name, null: false
  t.text :arguments
```

```
t.integer :priority, default: 0, null: false
t.string :active_job_id
t.datetime :scheduled_at
t.datetime :finished_at
t.string :concurrency_key
t.timestamps
end
```

Denne tabel bruges til at gemme information om alle background jobs.

Når et job er klar til at blive kørt, flyttes det videre til andre interne tabeller såsom:

```
create_table :solid_queue_ready_executions do |t|
  t.bigint :job_id, null: false
  t.string :queue_name, null: false
  t.integer :priority, default: 0, null: false
  t.datetime :created_at, null: false

  t.index :job_id, unique: true
  t.index [:priority, :job_id], name: "index_solid_queue_poll_all"
  t.index [:queue_name, :priority, :job_id], name:
    "index_solid_queue_poll_by_queue"
end
```

Solid Queue indeholder desuden flere andre tabeller til blandt andet:

- Failed jobs
- Completed jobs
- Jobs i kø
- Scheduled jobs

For at køre jobs startes en separat process med:

```
bin/jobs
```

Denne process fungerer som en worker-service, der konstant overvåger databasen for nye jobs.

I projektet blev workerne konfigureret således:

```

default: &default
  dispatchers:
    - polling_interval: 1
      batch_size: 500

  workers:
    - queues: "*"
      threads: 3
      processes: <%= ENV.fetch("JOB_CONCURRENCY", 1) %>
      polling_interval: 0.1

```

Her:

- Kan der maksimalt køres 3 threads samtidigt
- Workerne tjekker databasen 10 gange i sekundet
- Alle queues overvåges

Dette gjorde det muligt at køre flere background jobs samtidigt uden at påvirke brugeroplevelsen på hjemmesiden.

Recurring jobs

Solid Queue understøtter også recurring jobs gennem en recurring.yml fil.

Eksempel:

```

call_bot:
  class: CallBotJob
  schedule: every 1 minutes

```

Her ville Solid Queue automatisk oprette et nyt CallBotJob hvert minut.

Selve jobbet kunne eksempelvis se således ud:

```

class CallBotJob < ApplicationJob
  queue_as :default

  def perform

```

```
MarketBotService.perform

next_run = rand(1.minute..2.hours)

self.class.set(wait: next_run).perform_later
end
end
```

Når jobbet bliver kørt, kalder det automatisk perform-metoden.

I projektet blev recurring.yml dog ikke brugt direkte til marketplace-botten.

I stedet blev det valgt, at botten skulle starte første gang serveren blev startet. Herefter genstarter jobbet automatisk sig selv med et tilfældigt tidsinterval.

Dette blev gjort, fordi det gav mere naturlig bot-adfærd på marketplace-systemet. Hvis botten handlede med præcise faste intervaller, ville markedet virke mere kunstigt og forudsigeligt.

Jobbet kalder i praksis blot:

```
MarketBotService.perform
```

som herefter aktiverer marketplace-botten (Se afsnittet om Services).

Denne løsning gjorde det muligt at skabe mere dynamisk aktivitet på marketplace-systemet uden konstant manuel styring.

Engine(Backend - Daniel - Go):

Struktur:

Jeg vil indledningsvis lige fortælle om strukturen i projektet og forklarer hvorfor jeg har taget de valg jeg har taget i den sammenhæng.

Alt med Engine ligger i src/engine yderlig forklaring på dette kommer i afsnittet om CI/CD. I engine mappen findes der så cmd, db, handlers, models, routes. Derudover har vi en sum, mod og main.go fil. Den eneste grund til vi ligger en main.go fil her er fordi go ikke vil køre uden en main package i roden. Dog syntes jeg det giver dårlig struktur man kunne argumentere at det giver lige så fin mening at man konsekvent altid har en main i roden som starter programmet men for mig er det bedre at mappe stukture det derfor er det sådan. Så kommer den yderlige

indeling de bør være selv forklarende men vi nupper dem lige. CMD = her bliver programmet styret altså opstart osv man kan også se det lidt som maskinerummet hvor alt mødes og bliver koblet sammen.

DB = database så her ligger blot function til at etablere db connection og alt andet som ville være multipurpose functioner. Handlers = her ligger de handlers som bliver brugt på ruter, der ligger også nogle handlers som ikke er store nok til deres egen mapppe dog ville det nok være bedst og mest best pratice. Disse er tick, healthz og auth de ligger her fordi healthz teknisk er en handler til ruten healthz, auth bliver brugt i ruterne og tick er det mechanic som engine bruger. Models = er de data modeller der bliver brugt samt nogle små funktioner der bliver brugt i assosiation med modellerne. Routes er ruterne, man kunne også godt opdelle dette lidt bedre så det matcher lidt lige som i handlers mappen så hver kategori har en mappe og en handler som bliver bygget også en samlet der kombinere alle kategorier sammen i en route fil. Dog er dette projekt ikke stort nok til at det var tiden værd. Nogen kalder dette lokalt skill issue.

Opstart

Jeg vil lige kort gennemgå hvad der sker når programmet køres.

```
_ = godotenv.Load()

database, err := db.Connect()

if err != nil {

    log.Fatalf("failed to connect to database: %v", err)

}

port := os.Getenv("PORT")

if port == "" {

    port = "6969"

}
```

```

go handlers.RunTickLoop(database)

log.Printf("engine listening on 0.0.0.0:%s", port)

if err := routes.ListenAndServe(port, database); err != nil {
    log.Fatalf("server stopped: %v", err)
}

```

Så i ovenstående er alt der opstarter projektet, som tidligere forklaret er det lidt vandhullet hvor dyrene mødes og linker up og dette kan også ses direkte i koden. Første linje loader env filen ind i projektet så env filen kan tilgås gennem os pakken som kommer med go.

Efter vi har loadet env ind i context er vi klar til at forbinde til vores db så det gør vi ved at importere vores db pakke og kalde connect funktionen

```

database, err := db.Connect()

if err != nil {
    log.Fatalf("failed to connect to database: %v", err)
}

```

Vi logger sammentidig en fejl hvis err som bliver returneret fra connect funktionen er noget andet end nil. Det kan måske være lidt modsat men det er nemmere at sige hvis fejlen ikke er nil log end hvis vi skulle tjekke efter alle fejl typer selv, også bliver de specielle fejl returneret fra connect funktionen.

Dernæst henter vi den port vi gerne vil kører på, måske lidt bad practice at have en værdi hardcoded her men det er mere for at err tjekke mig selv i det tilfælde af jeg glemmer at lave env.

Dernæst starter vi vores tickloop op som ligesom er core functionalitet i spillet her kan man igen se hvordan cmd blot er et vandhul vi henter fra handlers pakken RunTickLoop som gør som navnet antyder kører tickloopet som er core mechanic af spillet. Det eneste der er værd at bemærke her er starten "go", dette betyder at functionen bliver kørt som en goroutine en goroutine er en "lightweight concurrent thread managed by the go runtime" Basicly er det en function der bliver managed i en thread af go som kører i baggrunden mens resten af programmet eksekveres.

```
go handlers.RunTickLoop(database)
```

Så skal vi også have nogle api endpoints på så Kim kan få lov at lave noget frontend så derefter starter vi vores ruter op

```
log.Printf("engine listening on 0.0.0.0:%s", port)

if err := routes.ListenAndServe(port, database); err != nil {
    log.Fatalf("server stopped: %v", err)
}
```

For x antal gang igen eksempel på at vi er ved vandhullet hvor mr.Mathias svømmer rundt, vi henter fra vores routes pakke at ruterne skal ListenAndServe samt giver vores db og port som parametere, vi definere err her fordi det som vi retunere fra ListenAndServe er en err også siger vi igen at hvis der er en fejl her skal den log ellers kører banditten bare.

DB

Jeg vil lige meget kort gennemgå hvordan vi forbinder til database og hvorfor gorm er gammel og nice. Så først henter vi vores connection string fra vores env bemærk her at vi henter fra

vores operation system i dette tilfælde er det context af programmet og ikke os som i windows, linux eller mac. Dette kan vi fordi vi henter vores env ind i os i vores cmd/server fil som forklaret tidligere. Dette gemmer vi som en variable så vi kan kalde det senere i go opretter kan man lave variable ved blot at smide ":" foran "=" så man slipper for f.eks at skrive var dsu = xyz, men syntax bliver lidt nemmere:

```
dsu := os.Getenv("DATABASE_URL")

if dsu == "" {

    log.Println("DATABASE_URL is not set")

    return nil, os.ErrNotExist

}
```

Dernæst opretter vi en variable til at hente vores db session, det er en indbygget function fra gorm som kræver en variable til sessionen og en til err så derfor har vi 2 "db, err". Vi giver den at det er en postgres db og vores connection string som vi hentede fra env før. Derefter kalder vi også gorms indbyggede function der returnere selve databasen og gemmer den som bdDB (bigDawgsDataBase).

```
db, err := gorm.Open(postgres.Open(dsu), &gorm.Config{})

if err != nil {

    return nil, err

}

bdDB, err := db.DB()

if err != nil {

    return nil, err

}
```

Så prøver vi at pinge denne for at tjekke der er hul igennem for hvis der ikke er så får vi fejlen her som så bliver returneret tilbage til vores vandhul så mr.mathias kan komme ud af det hvis der er krokodiller. Så migrater vi vores modeller ind i tabeller.

```
if err := bdDB.Ping(); err != nil {  
  
    return nil, err  
  
}  
  
if err := models.AutoMigrate(db); err != nil {  
  
    return nil, err  
  
}
```

Modeller:

Så jeg vil lige gennemgå kort hvordan vores modeller fungerer jeg vil ikke gå dem begge igennem men blot tage building modellen.

Så for det utrænede øje er der lidt en mærkelig struktur her i filen så vi går det igennem slavisk. I gode gamle js dage var best practice at have variable i toppen og functioner eller logik i bunden og det er efter det princip jeg kører efter her. Så vi starter med at oprette en type BuildingKey som er en string dette skal vi bruge i vores variable helvede til at oprette hver af vores bygnings typer som så kommer under.

```
type BuildingKey string  
  
  
  
  
  
  
  
  
const (  
  
    MeatFactory BuildingKey = "meat_factory"
```

```

    DogCoinDen    BuildingKey = "dog_coin_den"

    Doghouse      BuildingKey = "the_doghouse"

    DogKennel     BuildingKey = "the_dog_kennel"

    Market        BuildingKey = "market"

)

```

Så opretter jeg nogle multipler variabler så hvis vi skulle ændre dem gør vi det kun et sted.

```

const (

    ProductionMultiplier = 1.15

    CostMultiplier      = 1.25

)

```

Så opretter jeg en struct til at holde hvad produceres og her menes hvilken type resourcer.

```

type Production struct {

    DogCoins int64 `json:"dog_coins"`

    DogBones int64 `json:"dog_bones"`

    Dogs     int64 `json:"dogs"`

}

```

Så opretter vi en struct til at fortælle hvordan bygninger er defineret

```

type BuildingDefinition struct {

    Key          BuildingKey

}

```

```

DisplayName    string

Produces       string

BaseProduction Production

BaseCost       Production

BaseResourceCap Production

```

Her gør vi brug af vores tidligere typer oprette såsom production og buildingkey.

Dernæst laver jeg et map som så laver alle de typer bygninger vi gerne vil have med de start værdier som vi har valgt, dette gøre fordi når vi leveler et bygning op skal den have en start værdi så multiplieren fungere og hvis de ikke havde start værdier ville det være svært at få nogle resourcer.

```

var BuildingDefinitions = map[BuildingKey]BuildingDefinition{

    MeatFactory: {

        Key:          MeatFactory,

        DisplayName:   "Meat Factory",

        Produces:      string(ResourceDogBone),

        BaseProduction: Production{DogBones: 10},

        BaseCost:      Production{DogBones: 100, DogCoins: 50, Dogs: 5},

    },

    DogCoinDen: {

        Key:          DogCoinDen,

        DisplayName:   "Dog Coin Den",

        Produces:      string(ResourceDogCoin),

```

```

    BaseProduction: Production{DogCoins: 5},

    BaseCost:      Production{DogBones: 200, DogCoins: 100, Dogs: 10},

  },

  Doghouse: {

    Key:           Doghouse,

    DisplayName:   "The Doghouse",

    BaseCost:      Production{DogBones: 300, DogCoins: 150, Dogs: 10},

    BaseResourceCap: Production{DogBones: 1000, DogCoins: 500, Dogs: 20},

  },

  DogKennel: {

    Key:           DogKennel,

    DisplayName:   "The DogKennel",

    Produces:      string(ResourceDog),

    BaseProduction: Production{Dogs: 2},

    BaseCost:      Production{DogBones: 150, DogCoins: 75},

  },

  Market: {

    Key:           Market,

    DisplayName:   "The Market",

    BaseCost:      Production{DogBones: 500, DogCoins: 250, Dogs: 25},

  },

```

```
}
```

Dernæst opretter vi selve Building modellen som er det der faktisk bliver gemt i databasen. Her gør vi brug af `gorm.Model` som er en built-in struct fra `gorm` der giver os ID, `CreatedAt`, `UpdatedAt` og `DeletedAt` gratis. De tags du ser i backticks er `gorm` og `json` tags som fortæller `gorm` hvordan den skal mappe til databasen og `json` hvad feltet skal hedde når det sendes ud.

```
type Building struct {  
    gorm.Model  
  
    UserID      uint      `gorm:"uniqueIndex:idx_user_building_key;not  
null;index" json:"user_id"`  
  
    Key         string    `gorm:"uniqueIndex:idx_user_building_key;not null"  
json:"key"`  
  
    Level       int       `gorm:"not null;default:0"  
json:"level"`  
  
    IsConstructing bool      `gorm:"not null;default:false"  
json:"is_constructing"`  
  
    StartedAt   *time.Time `json:"started_at"`  
  
    CompletesAt *time.Time `json:"completes_at"`  
  
    DoghouseLevel int       `gorm:"-"  
json:"- "`  
}
```

`UserID` og `Key` har et `uniqueIndex` der hedder det samme, det betyder at kombinationen af de to skal være unik – altså en bruger kan ikke have to `meat_factory` bygninger. `DoghouseLevel` har `gorm:"-"` og `json:"-"` som betyder at `gorm` og `json` ignorerer feltet fuldstændigt, det er blot et hjælpefelt vi bruger internt i beregninger.

Så har vi en MarshalJSON funktion som overrumpler hvordan Go serialiserer en Building til JSON. Grunden til vi gør det er at vi ikke bare vil sende rå database data ud – vi vil også beregne og sende UpgradeCost, ProductionPerTick og ResourceCap med ud så frontend ikke selv skal regne det ud.

```
func (b Building) MarshalJSON() ([]byte, error) {

    type Alias struct {

        ID          uint      `json:"id"`

        CreatedAt   time.Time `json:"created_at"`

        UpdatedAt   time.Time `json:"updated_at"`

        UserID      uint      `json:"user_id"`

        Key         string    `json:"key"`

        Level       int       `json:"level"`

        Produces    string    `json:"produces"`

        IsConstructing bool      `json:"is_constructing"`

        StartedAt   *time.Time `json:"started_at"`

        CompletesAt *time.Time `json:"completes_at"`

        UpgradeCost Production `json:"upgrade_cost"`

        ProductionPerTick Production `json:"production_per_tick"`

        ResourceCap Production `json:"resource_cap"`

    }

    def := BuildingDefinitions[BuildingKey(b.Key)]

    return json.Marshal(Alias{

        ID:          b.ID,
```

```

        CreatedAt:      b.CreatedAt,

        UpdatedAt:      b.UpdatedAt,

        UserID:         b.UserID,

        Key:            b.Key,

        Level:          b.Level,

        Produces:        def.Produces,

        IsConstructing:  b.IsConstructing,

        StartedAt:       b.StartedAt,

        CompletesAt:     b.CompletesAt,

        UpgradeCost:     b.UpgradeCost(),

        ProductionPerTick: b.ProductionPerTick(),

        ResourceCap:     ResourceCap(b.DoghouseLevel),

    })
}

```

Dernæst er der IsValidBuildingKey som blot tjekker om en given key eksisterer i vores BuildingDefinitions map og returnerer sand eller falsk. Bruges til at validere bruger input.

```

func IsValidBuildingKey(key string) bool {

    _, ok := BuildingDefinitions[BuildingKey(key)]

    return ok

}

```

Så har vi `ProductionPerTick` som beregner hvor meget en bygning producerer per tick baseret på dens level. Her bruger vi vores `ProductionMultiplier` konstant og `math.Pow` til at udregne en eksponentiel stigning — jo højere level, jo mere produktion.

```
func (b Building) ProductionPerTick() Production {  
  
    definition, ok := BuildingDefinitions[BuildingKey(b.Key)]  
  
    if !ok || b.Level < 1 {  
  
        return Production{}  
  
    }  
  
    multiplier := math.Pow(ProductionMultiplier, float64(b.Level-1))  
  
    return Production{  
  
        DogCoins: int64(math.Round(float64(definition.BaseProduction.DogCoins) *  
multiplier)),  
  
        DogBones: int64(math.Round(float64(definition.BaseProduction.DogBones) *  
multiplier)),  
  
        Dogs:      int64(math.Round(float64(definition.BaseProduction.Dogs) *  
multiplier)),  
  
    }  
}
```

`ResourceCap` beregner hvor mange ressourcer en bruger max kan have baseret på deres Doghouse level — jo højere doghouse, jo mere kan man opbevare. Er doghouse level 0 returneres en tom `Production` så man ikke kan opbevare noget.

Til sidst UpgradeCost som beregner hvad det koster at opgradere en bygning til næste level. Her bruger vi CostMultiplier på samme måde som produktionen – eksponentiel stigning jo højere level.

```
func ResourceCap(doghouseLevel int) Production {  
  
    def := BuildingDefinitions[Doghouse]  
  
    if doghouseLevel < 1 {  
  
        return Production{}  
  
    }  
  
    return Production{  
  
        DogCoins: def.BaseResourceCap.DogCoins * int64(doghouseLevel),  
  
        DogBones: def.BaseResourceCap.DogBones * int64(doghouseLevel),  
  
        Dogs:     def.BaseResourceCap.Dogs * int64(doghouseLevel),  
  
    }  
}  
  
func (b Building) UpgradeCost() Production {  
  
    definition, ok := BuildingDefinitions[BuildingKey(b.Key)]  
  
    if !ok {  
  
        return Production{}  
  
    }  
  
  
    multiplier := math.Pow(CostMultiplier, float64(b.Level))
```

```

    return Production{

        DogCoins: int64(math.Ceil(float64(definition.BaseCost.DogCoins) *
multiplier)),

        DogBones: int64(math.Ceil(float64(definition.BaseCost.DogBones) *
multiplier)),

        Dogs:      int64(math.Ceil(float64(definition.BaseCost.Dogs) *
multiplier)),

    }
}

```

Ruter

Så jeg vil gennemgå hvordan ruterne er sat op også kan vi derefter kigge på en handler. Det meste der associeres med ruterne ligger i mappen routes. Som forklaret tidligere ville det være fordelagtigt at have det opdelt her så hver kategori har deres egen mappe også have en main fil til at forbinde alle de forskellige kategories ruter. Dog er der lige på grænsen her i projektet til hvad jeg vil mene er nødvendigt for at gøre dette men hvis man skulle udvide projektet ville det være et naturligt første skridt at tage og nogle nemme overskuds point og hente. Lige så meget for at køre lidt mere SoC, men hvis vi tjekker inde i mappe definere vi først vores auth, denne auth er til at sikre at vores stier kun kan rammes med en gyldig token, jeg kommer ikke til at gennemgå jwt koden her i dybden men det korte og det lange er at vi tjekker om der er en auth header på og om dette er en valid token. Den eneste rute hvor dette tjek ikke findes er healthz ruten da dette blot er til at tjekke om serveren er oppe og kører, dette må folk uden token godt vide da de ikke kan få noget brugbart eller sårbart info ud af denne.

```

auth := handlers.RequireAuth

mux.HandleFunc("/healthz", handlers.HealthzHandler)

```

Selve Register funktionen er der hvor alle ruterne bliver koblet op. Vi starter med at gemme handlers.RequireAuth som en lokal variabel auth – dette er blot for at gøre koden kortere at læse, i stedet for at skrive handlers.RequireAuth(...) på hver linje skriver vi blot auth(...).

```
mux.Handle("GET /buildings", auth(buildings.ListBuildingsHandler(db)))
```

Her ser vi mønsteret – vi definerer HTTP metoden og stien, wrapper handleren i auth og giver databasen med som parameter.

Ruterne er opdelt i tre kategorier: buildings, resources og transactions – hver med deres egne handlers importeret fra deres respektive undermapper i handlers.

mux er kort for multiplexer, i Go's standard bibliotek er dette http.ServeMux og det er basicly en router – den lytter på indkommende HTTP requests og sender dem videre til den rigtige handler baseret på stien.

Til sidst er der ListenAndServe som er det der faktisk starter serveren. Den opretter en ny mux, registrerer alle ruterne via Register og starter HTTP serveren på den port vi definerede i cmd.

```
func ListenAndServe(port string, db *gorm.DB) error {  
  
    mux := http.NewServeMux()  
  
    Register(mux, db)  
  
    addr := fmt.Sprintf("0.0.0.0:%s", port)  
  
    return http.ListenAndServe(addr, mux)  
}
```

Handlers

Jeg vil ikke gennemgå alle handlers da de fleste af dem følger det samme mønster, jeg vil gennemgå en enkelt herunder og mange af de samme mønstre gør sig gældende i dem alle.

Så strukturen fungerer således at vi toppen definere en type som er den besked vi gerne vil sende når funktionen har eksekveret. Dette bør være skrevet ind i en model så vi kunne kalde den samme hvergang så her er igen hurtige overskuds point og hente hvis man overtager projektet.

```

type GetBagResponse struct {
    Message      string                `json:"message"`
    ResourceBags map[string]models.ResourceBag `json:"resourcesBag"`
}

```

Dernæst kommer selve funktionen som rent faktisk bliver kørt når ruten rammes,

Vi starter med at bruge go's indbyggede http pakke til at sige den her "normale" function skal bruges på dette http kald. Derefter bruger vi vores function fra auth denne function returnere et uid og en fejl og som altid i go tjekker vi efter om fejlen er andet end nil hvis den er andet end nil returnere den en fejl.

```

    userID, err := handlers.UserID(r)

    if err != nil {
        http.Error(w, "missing authenticated user", http.StatusUnauthorized)

        return
    }

```

Dette uid bruger vi så til at slå op i db om der findes en entry med dette uid associeret.

```

    var resourceBags []models.ResourceBag

    if err := db.Where("user_id = ?", userID).Find(&resourceBags).Error; err != nil {
        http.Error(w, "failed to load resource bag", http.StatusInternalServerError)

        return
    }

```

Derefter bruger vi dataen fra db til at lave et object af med de forskellige resourcer som så matcher den model vi har bygget. Dette giver så et object med flere objecter dette er lavet således fra request af Kim fordi det var bedre for ham at have det på denne måde og i sidste ende har kunden altid ret på den ene eller anden måde, selv når de tager fejl .

```
"resourcesBag": {  
  
  "dogbones": {  
  
    "ID": 38,  
  
    "CreatedAt": "2026-05-06T10:26:52.67575Z",  
  
    "UpdatedAt": "2026-05-06T11:28:47.95007Z",  
  
    "DeletedAt": null,  
  
    "User_id": 19,  
  
    "ResourceKey": "dogbones",  
  
    "Amount": 1000  
  
  },  
  
  "dogcoin": {  
  
    "ID": 37,  
  
    "CreatedAt": "2026-05-06T10:26:52.667278Z",  
  
    "UpdatedAt": "2026-05-12T09:42:13.87509Z",  
  
    "DeletedAt": null,  
  
    "User_id": 19,  
  
    "ResourceKey": "dogcoin",  
  
    "Amount": 500  
  
  },  
  
}
```

```

    "dogs": {

        "ID": 39,

        "CreatedAt": "2026-05-06T10:26:52.677612Z",

        "UpdatedAt": "2026-05-12T09:42:13.876082Z",

        "DeletedAt": null,

        "User_id": 19,

        "ResourceKey": "dogs",

        "Amount": 20

    }

}

```

Så smider vi en header på til response tilbage og at status var ok samt vores beksed ud efter den type vi definerede i toppen.

```

bagMap := make(map[string]models.ResourceBag, len(resourceBags))

for _, r := range resourceBags {

    bagMap[r.ResourceKey] = r

}

w.Header().Set("Content-Type", "application/json")

w.WriteHeader(http.StatusOK)

_ = json.NewEncoder(w).Encode(GetBagResponse{

    Message:      "resource bag loaded",

```

```
ResourceBags: bagMap,  
  
}))
```

understregen `__` betyder vi bevidst ignorerer den fejl `Encode` potentielt kan returnere. På dette tidspunkt i koden er headeren og statuskoden allerede skrevet til response med `w.WriteHeader`, så vi kan ikke sende en ny fejl til klienten alligevel – derfor giver det ikke mening at håndtere den.

Tick systemmet

Tick systemet er core mechanic i spillet – det er det der sørger for at ressourcer bliver produceret over tid. `RunTickLoop` kører som den goroutine vi startede i `cmd` og fyrer et tick af hvert 15. sekund via Go's indbyggede `time.NewTicker`.

```
ticker := time.NewTicker(15 * time.Second)  
  
for range ticker.C {  
  
    processTick(db)  
  
}
```

Selve logikken ligger i `processTick` som gør tre ting:

1. Færdiggør bygninger – den tjekker om der er bygninger under konstruktion hvor `completes_at` er passeret, og rykker dem et level op.
2. Beregner produktion – for hver bruger henter den alle bygninger, finder doghouse levellet for at kende resource cap, og summer den samlede produktion op på tværs af alle bygninger via `ProductionPerTick()` som vi gennemgik i modellen.
3. Opdaterer resourcer – den bruger `LEAST(amount + ?, ?)` direkte i SQL-kaldet, det betyder databasen selv sørger for at man aldrig går over sin cap atomically uden at vi behøver hente værdien ned og opdatere den i to operationer.

CI/CD(DevOps - Daniel - Dokploy):

Vi har som i et hvert andet mediocore kode projekt opsat en form for ci/cd samt hosting, vi har valgt at bruge dokploy og låne et domæne ved Mathias som kører gennem en cloudflare tunnel.

Den måde som dokploy fungerer på er at vi har en server ved skolen som kører dokploy mest for at vi godt kan lide at spare vores egen strømregning og derfor gerne vil slukke vores pc engang i mellem. På denne maskine har vi så installeret dokploy, det har vi gjort efter anvisningerne i deres doc.

```
curl -sSL https://dokploy.com/install.sh | sh
```

Dette er kørt på vores maskine, som så åbner for et web interface som hvor vi så opretter en admin bruger til vores projekt i mellemtiden har vi fået en port-forwarder vi til cloudflare så port:3000 bliver til <https://bigdawgs.mags.dk/>

Her har vi så lavet en bruger pr mand (bilag 10), så vi alle kan tilgå og tjekke logs. Så laver vi et nyt projekt inde i dokploy (bilag 11 + 12) i det projekt opretter vi en ny service (bilag 13) af typen compose (bilag 14), det er til at pege på en compose fil som ligesom er servicen (bilag 15). Denne compose har i vores tilfælde en dockerfile pr service til opstart samt opsætning af env.

```
engine:

  build:

    context: ./engine

    dockerfile: Dockerfile

  environment:

    PORT: ${ENGINE_PORT}

    DATABASE_URL: ${DATABASE_URL}

    JWT_SECRET: ${JWT_SECRET}

  ports:

    - "6969:6969"
```

```
depends_on:

  postgres:

    condition: service_healthy
```

Det er en af grundene til at vores repo har den struktur med src også alle filerne i src. Så engine ligger i src, api ligger i src og frontend gør også (bilag 16). Tænker ikke umiddelbart at vi vil gå alt for meget i dybden med vores compose fil det er dog værd at bemærke at vi i vores compose service definere vores env og bruger disse i compose så vi slipper for at skal dele env filer mellem hinanden eller manuelt skrive dem så kan man bare gå på dokploy og hente værdier hvis det er nødvendigt. Vi opsætter så derefter vores service til at pege på vores github repo og peger på vores compose fil også gør dokploy egentlig resten for os selv (bilag 17), det er ret basic setup men fungerer skide godt og nemt. Her kan vi så vælge hvornår den skal køre igen vi har den til at køre på push, vi kan som tidligere nævnt sætte vores env ind som skal bruges i compose direkte ind. Desuden er det så smart vi også her kan tilgå logs fra hver af de containers der bliver startet, hver service har deres egen container så vi kan nemt fejlfinde hvis noget går galt i f.eks engine kan jeg tjekke logs fra engine containeren.

Derefter har vi igen lånt Mr.Mathias cloudflare og portwarder vores valgte porte til url's i cloudflare så vores api bliver

<https://bd-frontend.mags.dk/>

<https://bd-engine.mags.dk/>

<https://bd-economy.mags.dk/>

API(Backend - Kevin - C#):

Introduktion

I dette projekt har jeg arbejdet med udviklingen af en economy-service i ASP.NET Core. Servicen håndterer et markedssystem, hvor brugere kan købe og sælge ressourcen DogCoins. Prisen ændrer sig dynamisk ud fra udbud og efterspørgsel.

Backendens formål er at stille et sikkert API til rådighed, hvor frontend kan:

- hente aktuelle priser
- udføre handler
- simulere bot-handler
- hente historik over tidligere handler

Overordnet struktur

Economy-servicen er opbygget som en REST API-service i ASP.NET Core. Projektet er struktureret efter en lagdelt arkitektur, hvor ansvaret er opdelt mellem controllers, services, DTO'er og tests.

Formålet med denne struktur er at skabe en tydelig separation of concerns, hvor hver del af systemet har ét specifikt ansvar. Dette gør koden mere overskuelig, lettere at vedligeholde og nemmere at udvide.

Projektets struktur består blandt andet af:

- **Controllers** – håndterer HTTP requests og responses
- **Services** – indeholder markedslogik og forretningsregler
- **DTOs** – styrer data mellem frontend og backend
- **Tests** – indeholder unit tests af systemets funktionalitet

Denne opdeling gør det muligt at udvikle de enkelte dele uafhængigt af hinanden.

API-design og REST-principper

API'et er designet efter REST-principper, hvor endpoints er opdelt efter ansvar og funktionalitet.

Eksempelvis anvendes:

- **GET** til hentning af data
- **POST** til udførelse af trades

Dette gør API'et mere konsistent og lettere at forstå for frontend-udviklere.

Routes er samtidig struktureret omkring markedets funktioner:

- `/market/prices`
- `/market/trade`
- `/market/history`
- `/market/bot/trade`

Denne struktur gør API'et overskueligt og selvforklarende.

Service-lag og forretningslogik

En vigtig designbeslutning har været at placere al markedslogik i `MarketService` fremfor direkte i controlleren.

Controllerens ansvar er kun at modtage requests og returnere responses, mens al beregning af priser, supply/demand og trade history håndteres i service-laget.

Dette gør systemet lettere at teste, fordi logikken kan testes uden HTTP eller controllers. Samtidig gør det controllerne mere simple og overskuelige.

Eksempelvis sendes trade-requests videre til servicen:

```
return Ok(_marketService.HandleTrade(request, userId));
```

Denne struktur gør det lettere senere at udvide systemet med flere controllere eller andre klienter uden at ændre markedslogikken.

Inputvalidering og robusthed

Der er lagt vægt på inputvalidering for at sikre, at ugyldige trades ikke påvirker markedet.

Systemet afviser blandt andet:

- ugyldige trade types
- negative værdier
- zero amount

Dette testes også gennem unit tests.

Ved at validere input tidligt reduceres risikoen for fejl og inkonsistente markedsddata.

Supply og demand model

Markedssystemet er bygget op omkring en simpel supply/demand model, hvor prisen på DogCoins ændres dynamisk ud fra køb og salg.

Ved køb:

- demand stiger
- supply falder
- prisen stiger

Ved salg:

- supply stiger
- demand falder
- prisen falder

Formålet med modellen er at skabe et marked, som reagerer dynamisk på brugeraktivitet fremfor faste priser.

Prisberegningen tager blandt andet højde for:

- trade amount
- market pressure
- tidligere pris
- minimums- og maksimumspriser

Der anvendes samtidig `Math.Clamp`, så markedet ikke bliver ustabil eller producerer urealistiske værdier.

```
return Math.Round(  
    Math.Clamp(newPrice, minPrice, maxPrice),  
    2);
```

Dette sikrer, at markedet forbliver balanceret.

Performance og memory-overvejelser

Da markedet kører som en singleton-service i memory, er der lagt vægt på at holde beregninger og datastrukturer simple og hurtige.

Alle trades håndteres direkte i memory uden databasekald, hvilket gør svartiden meget lav. Dette gør løsningen velegnet til hurtige markedsopdateringer, hvor mange handler kan ske på kort tid.

Samtidig betyder dette design, at markedets tilstand nulstilles, hvis servicen genstartes. Dette er accepteret i projektets nuværende scope, men kunne i en produktionsløsning løses ved at gemme markedets data i en database eller distributed cache.

Singleton design og thread-safety

`MarketService` er registreret som singleton i applikationen:

```
builder.Services.AddSingleton<MarketService>();
```

Dette betyder, at hele applikationen deler samme markedsinstans og dermed samme økonomi.

Fordelen ved dette design er, at alle brugere påvirker det samme marked i realtid. Hvis én bruger køber DogCoins, ændres prisen for alle andre brugere.

Da flere requests kan ramme servicen samtidigt, er der implementeret thread-safety ved brug af `lock`.

```
lock (_lock)
{
    // market logic
}
```

Dette sikrer, at supply, demand og pris ikke ændres samtidigt af flere threads på en måde, som skaber inkonsistente data.

Skalering og videreudvikling

Systemet er designet, så funktionalitet kan udvides uden større ændringer i den eksisterende struktur.

Eksempelvis ville det være muligt senere at:

- tilføje flere ressourcer end DogCoins
- implementere flere markeder
- gemme trade history i database
- tilføje websocket-opdateringer i realtime
- implementere mere avancerede bots

Da logikken er placeret i service-laget, kan disse udvidelser implementeres uden større ændringer i controllerne.

API og sikkerhed

`MarketController` er beskyttet med `[Authorize]`, hvilket betyder, at brugeren skal være logget ind med en gyldig JWT-token for at bruge markedets endpoints.

Brugerens id hentes direkte fra tokenet:

```
return User.FindFirstValue(ClaimTypes.NameIdentifier)
    ?? User.FindFirstValue("sub")
    ?? User.FindFirstValue("id")
    ?? User.FindFirstValue("user_id");
```

Dette gør, at backend selv identificerer brugeren i stedet for at stole på data sendt fra frontend. Det forbedrer sikkerheden og forhindrer, at brugeren kan handle på vegne af andre.

Controlleren indeholder endpoints til:

- hentning af aktuelle priser
- udførelse af trades
- simulering af bot-handler
- hentning af trade history

BotService

Jeg har også arbejdet med **BotService**, som simulerer handler fra en bot.

Botten vurderer den nuværende pris og vælger, om den skal købe eller sælge. Hvis prisen er lav, er der større sandsynlighed for, at botten køber. Hvis prisen er høj, er der større sandsynlighed for, at botten sælger.

```
if (currentPrice <= 4m)
{
    type = Random.Next(100) < 75 ? "buy" : "sell";
}
else if (currentPrice >= 8m)
{
    type = Random.Next(100) < 75 ? "sell" : "buy";
}
```

Dette gør markedet mere dynamisk og realistisk.

DTO'er og API-design

Systemet anvender DTO'er (Data Transfer Objects) til kommunikation mellem frontend og backend.

DTO'erne bruges for at:

- kontrollere hvilke data frontend modtager
- undgå eksponering af interne modeller
- skabe et stabilt API-format
- gøre systemet lettere at versionere

Eksempelvis indeholder trade-request DTO'en kun:

- trade type
- amount

```
public string Type { get; set; } = string.Empty;
public int Amount { get; set; }
```

Dette gør API'et simpelt og tydeligt at anvende.

Swagger og udviklingsværktøjer

Swagger er integreret i projektet for at gøre det lettere at teste og dokumentere API'et under udvikling.

Dette gør det muligt hurtigt at:

- teste endpoints
- validere request/response strukturer
- verificere JWT-beskyttede routes

Swagger forbedrer udviklingsoplevelsen og gør API'et lettere at arbejde med under debugging og integration.

Testing strategi

Der er implementeret unit tests til både `MarketService` og `BotService`.

Formålet med testene er at verificere:

- at markedslogikken fungerer korrekt
- at køb øger prisen
- at salg sænker prisen
- at minimums- og maksimumspriser overholdes
- at ugyldige trades afvises
- at bot-handler påvirker markedet korrekt

Eksempelvis testes det, at køb får markedsprisen til at stige:

```
Assert.True(result.Resources.CurrentDogCoinsPrice > before);
```

Der testes også edge cases som:

- ugyldige trade types
- zero amount

- maksimum/minimum pris
- deterministisk adfærd

Denne tilgang gør det muligt at validere markedsløgikken automatisk og reducerer risikoen for fejl ved videreudvikling.

Refleksion over testing

Test-strategien har været fokuseret på markedsløgikken, da denne del er central for systemets funktionalitet.

Der er især lagt vægt på:

- edge cases
- prisgrænser
- validering
- deterministisk adfærd

Ved at teste både normale scenarier og ekstreme tilfælde bliver systemet mere robust overfor fejl og uventet input.

Refleksion over implementeringen

En styrke ved løsningen er den tydelige opdeling mellem API, markedsløgik og tests. Dette gør systemet lettere at vedligeholde og udvide.

Singleton-designet gør det muligt at simulere et fælles marked for alle brugere, hvilket passer godt til formålet med economy-servicen.

En mulig forbedring kunne være at flytte markedets data til en database i stedet for memory-storage, så markedets tilstand kunne overleve genstart af servicen.

Mit bidrag

I denne del af projektet har jeg arbejdet med economy-servicen, herunder:

- API-endpoints

- JWT-baseret adgangskontrol
- DTO-struktur
- markedslogik
- bot-simulering
- unit tests

Fokus har været at skabe en simpel, men dynamisk economy-service, hvor prisen på DogCoins ændrer sig ud fra brugerens og botten handlinger.

Mangler:

Rails

Selvom projektet nåede langt, er der stadig flere funktioner og forbedringer, som ikke nåede at blive implementeret inden projektets afslutning.

En af de største mangler er, at der endnu ikke findes direkte interaktion mellem spillerne udover det dynamiske marketplace-system. Det var blandt andet planen, at spillere skulle kunne:

- Angribe hinanden
- Udspionere andre byer
- Føre krig mod andre spillere

Derudover mangler der stadig et clan-system, som skulle gøre det muligt for spillere at samarbejde i grupper og skabe større multiplayer-interaktion.

Projektet mangler også automatiske tests på Rails/frontend-delen. Dette betyder, at meget funktionalitet stadig testes manuelt, hvilket ikke er optimalt i større projekter.

Derudover er der flere dele af koden, som med fordel kunne refaktoreres og struktureres bedre. Dette gælder især dele af frontend-logikken og enkelte services, som blev udviklet hurtigt under projektets udvikling.

Map-systemet er også meget begrænset i sin nuværende form. Kortet understøtter kun omkring 100 spillere ad gangen gennem et fast grid-system. Den oprindelige plan var, at kortet automatisk skulle kunne udvide sig dynamisk efter behov, men dette nåede ikke at blive implementeret.

Engine:

Den helt store mangel der findes i engine lige nu er helt klart at konvertere til at bruge websockets, samt at implementere en form for que system personligt ville jeg anbefale at bruge pubsub, fordi det er lightweight og gratis samt nemt at sætte op og det kan startes fra en dockerfile så det kan kobles sammen med compose og køres fra dokploy. Disse ting er ikke implementeret i en blanding på evner og tid, det ville egentlig godt have været muligt at få websockets med men vi valgte det fra imod at komme mere i mål som en decideret mvp og fordi det nuværende system teknisk virker og stadig vil skalere, dog ville det være muligt at sparere nogle kald da man nemt kan regne 1000xkald pr 15 sec bliver en del men på et mvp stadige er dette ikke et problem.

Desuden kunne man godt lave nogle flere opdelings ting som bliver nævnt tidligere i rapporten, det handler mest om at optimere SoC og gøre tingene mere maintainable og afhængigt af devven give en bedre dev oplevelse.

API

Selvom economy-servicen fungerer efter hensigten i de fleste scenarier, er der stadig områder, som kan forbedres.

En udfordring i den nuværende implementering er markedslogikken omkring køb og salg. Formålet med systemet er, at prisen skal stige ved køb og falde ved salg ud fra supply og demand. Under testing blev dette verificeret gennem unit tests, men i praksis oplevedes der scenarier, hvor prisen stadig kunne stige ved salg. Dette indikerer, at prisalgoritmen stadig kan forbedres og finjusteres for at skabe mere realistiske markedsudsving.

Derudover er markedets tilstand i øjeblikket gemt direkte i memory gennem singleton-services. Dette gør løsningen hurtig og simpel, men betyder samtidig, at alle data nulstilles, hvis servicen genstartes. I en mere avanceret løsning ville det være relevant at gemme markedsdata og trade history i en database.

Bot-systemet er også relativt simpelt og baseret på tilfældige sandsynligheder. Selvom dette skaber aktivitet på markedet, kunne botterne gøres mere avancerede ved at analysere historiske prisændringer eller reagere på bestemte markedstendenser.

En anden mulig forbedring ville være realtime-opdateringer via WebSockets. I den nuværende løsning skal frontend selv hente nye priser gennem API-calls. Med realtime-kommunikation ville prisændringer kunne vises direkte til alle brugere med det samme.

Derudover kunne systemet udvides med:

- flere ressourcer end DogCoins
- avanceret markedsanalyse

Samlet set er economy-servicen designet som et simpelt og fleksibelt fundament, der senere kan videreudvikles til et mere avanceret økonomi- og handelssystem.

Konklusion:

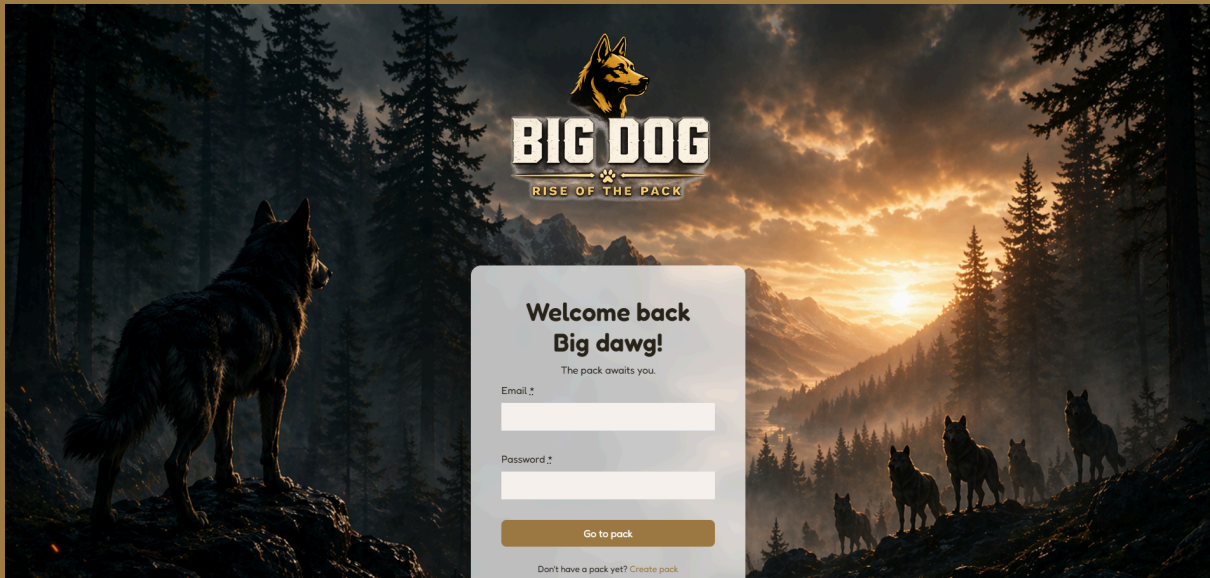
I dette projekt har vi formået at udvikle en webbaseret tick based RTS game, der giver brugeren en nem og god brugerflade som burgeren kan benytte til at lave sig en dawg pack og udradere sine modstandere. Gennem projektet har vi arbejdet med flere forskellige teknologier, herunder Ruby on Rails, Go, C#, PostgreSQL og dokploy. Dette har givet os en endnu bedre forståelse for hvordan man opbygger et system med flere komponenter, der skal arbejde sammen.

Vi har opnået de vigtigste mål med projektet, og fået en fungerende løsning og et lækkert design der er dog stadig massere at tage fat på såsom det med at have nogle npc'er der kan spille som modspillere nogle flere bygninger og nogle forskellige kombinationer og måske nogle former for racer der kan noget forskelligt og giver spilleren mulighed for at bevæge sig i forskellige retninger.

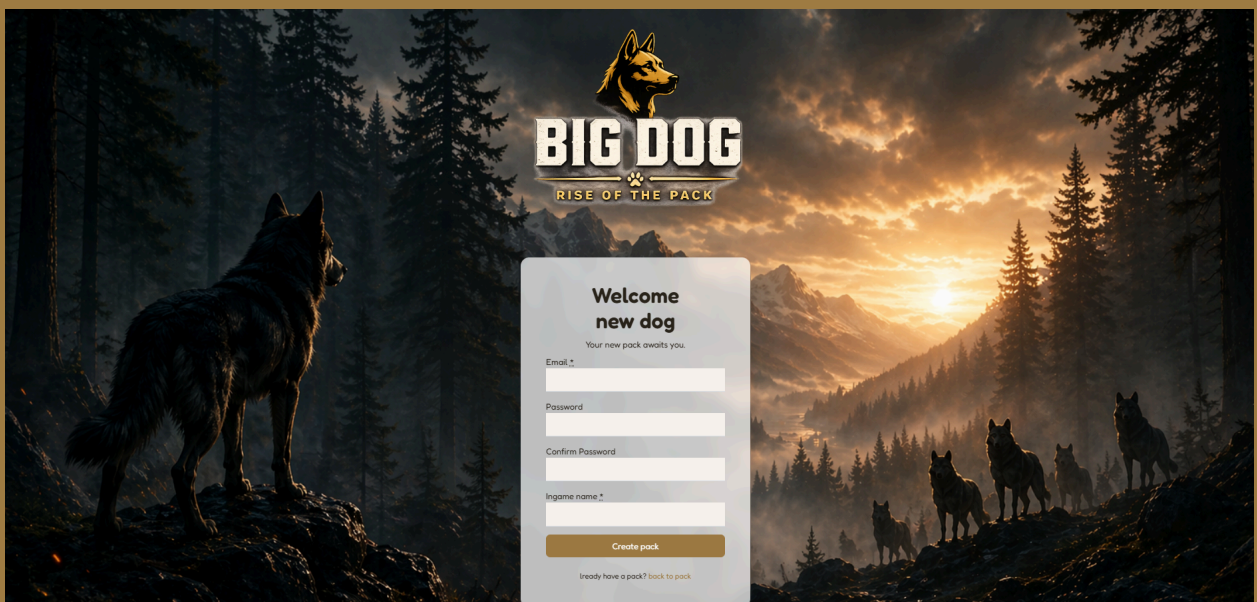
Samlet set har projektet givet os erfaring med både udvikling, opsætning og drift af et system og spil. Det har givet os indblik i hvordan man kordinere og organisere sig i et projekt og hvordan spil verden er anderledes end web verden selvom det er webbaseret.

Bilagsliste:

Bilag 1



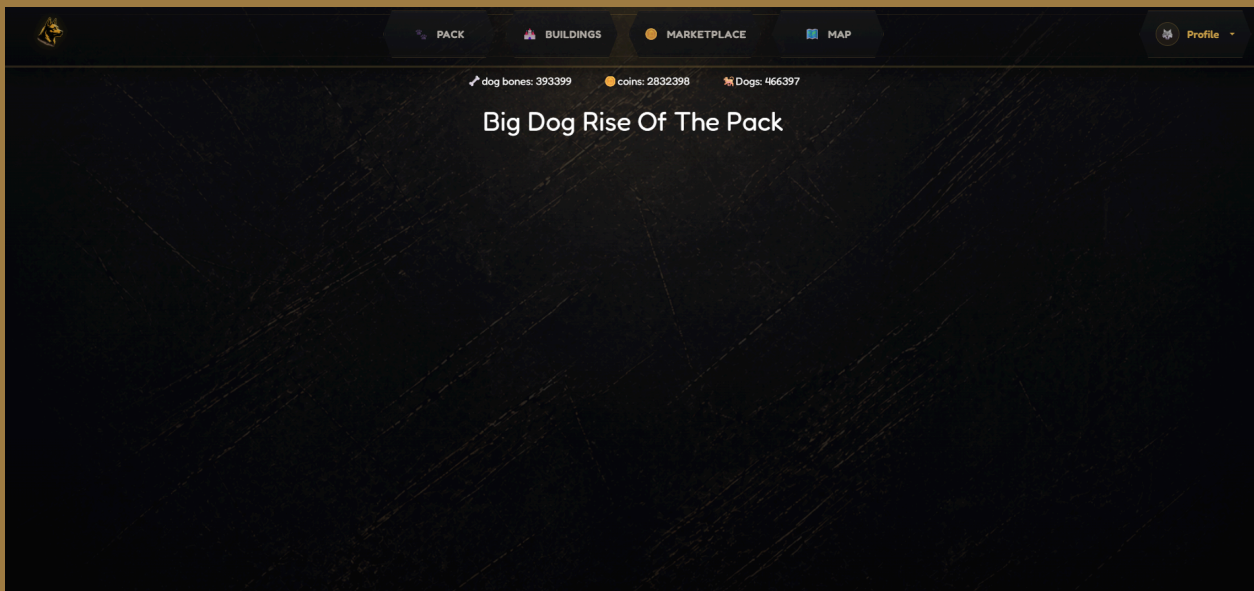
Bilag 2



Billag 3



Billag 4



Billag 5



Billag 6

 dog bones: 0/500

 coins: 0/1000

 Dogs: 0/20

Billag 7

Buildings to improve your pack

Market

Level 16

Building description

"Trade, thrive, dominate."

The Market connects your pack to the outside world. Trade resources, optimize your economy, and turn surplus into profit.

👉 Enables resource trading



Bones 27756 Coins 13878 Dogs 1388

[show details](#)

Dog bone factory

Level 6

Building description

"Where bones become power."

Your pack can't survive on loyalty alone. The Dog Bone Factory produces a steady supply of Dog bones, fueling your dogs and keeping them strong, focused, and ready to grow.

👉 Produces: Dog bones

👉 Higher level → higher production



Current production: 27 dog bones per tick

Bones 597 Coins 299 Dogs 30

[show details](#)

Dog coin den

Level 8

Building description

"Where the real deals are buried."

Not everything here is strictly legal—but it works. The Dog Coin Den generates coins through trade, deals, and maybe a few shady operations behind the scenes.

👉 Produces: Coins

Current production: 13 dog coins per tick



Bones 1193 Coins 597 Dogs 60

[show details](#)

The doghouse

Level 7

Building description

"The heart of the pack."

The Doghouse is your main base and the center of your pack's power. From here, you expand your territory, unlock new buildings, and control your progression.

👉 Sets max level for other buildings

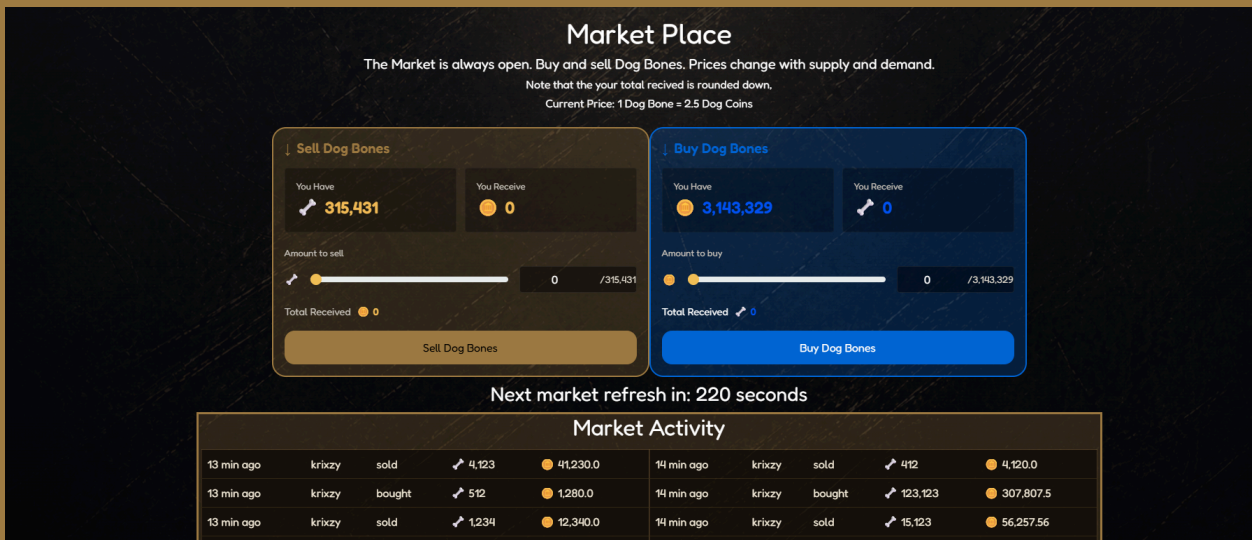
👉 Increases overall capacity



Bones 1431 Coins 716 Dogs 48

[show details](#)

Billag 8



billag 9



Bilag 10:

 **Sign Up**

Enter your email and password to create an account

 Sign in with GitHub

 Sign in with Google

Or register with email

First Name
John

Last Name
Doe

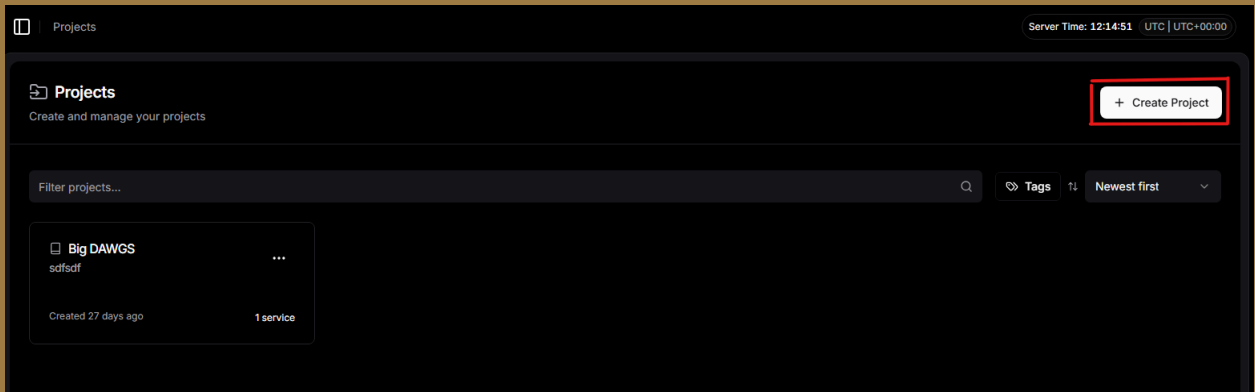
Email
email@dokploy.com

Password
Password 

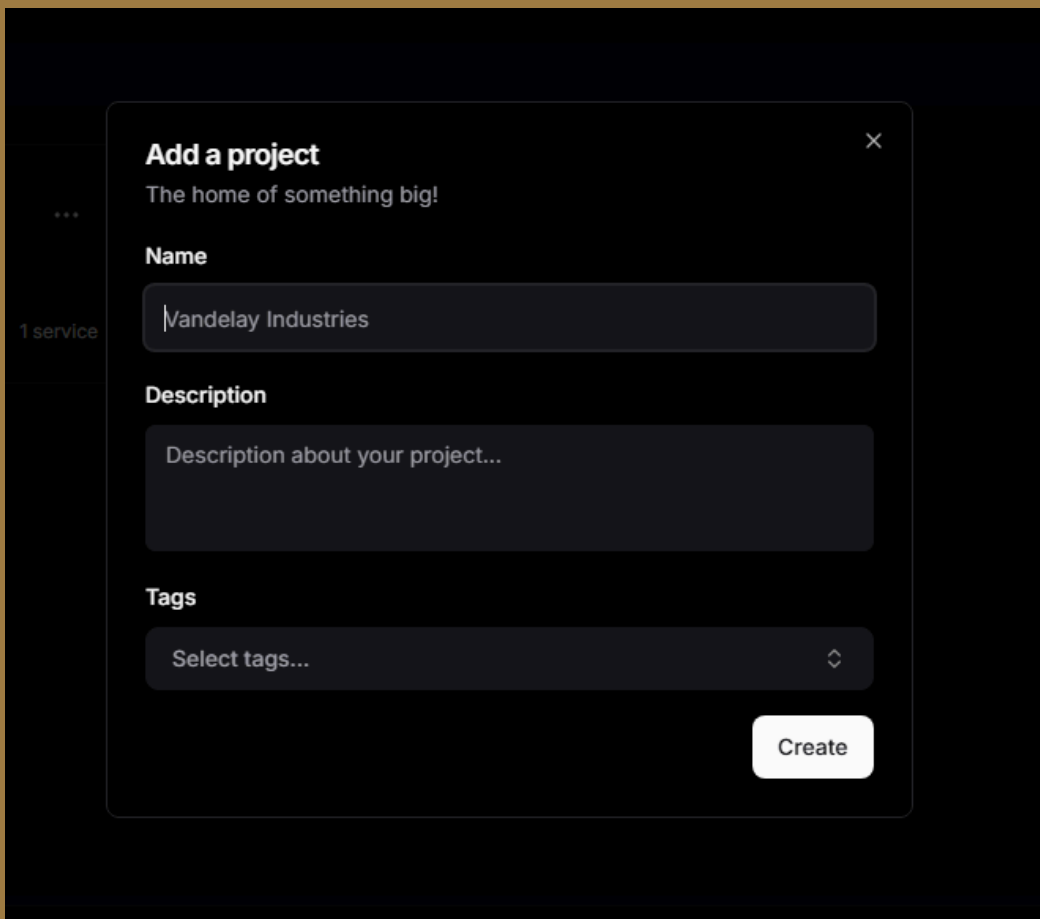
Confirm Password
Password 

Register

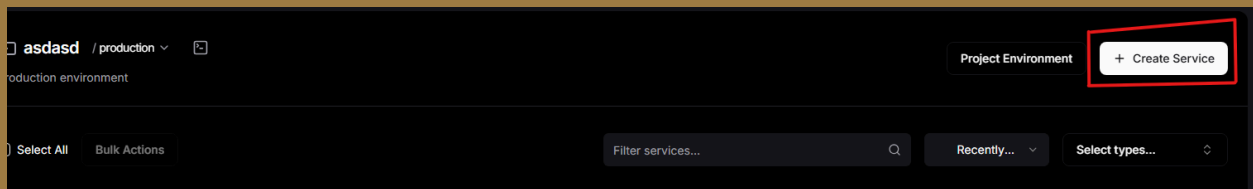
Bilag 11:



Bilag 12:



Bilag 13:



Bilag 14:

The screenshot shows a 'Create Compose' modal form with a close button (X) in the top right corner. The form has the following fields:

- Name:** A text input field containing 'sdfdsf'.
- App Name:** A text input field containing 'asdasd-sdfdsf'.
- Compose Type:** A dropdown menu with 'Docker Compose' selected and a downward arrow.
- Description:** A large text area with the placeholder text 'Description of your service...'.

A 'Create' button is located at the bottom right of the form.

Bilag 15:

Deploy Settings

Create a compose file to deploy your compose

DeployReloadStartOpen Terminal

Autodeploy ☐

Provider

Select the source of your code

Preview Compose

GitHub

GitLab

Bitbucket

Git

GitLab

Git

Raw

GitHub Account

Select a Github Account

Repository

Select repository

Branch

Select branch

Compose Path

./docker-compose.yml

Trigger Type

On Push

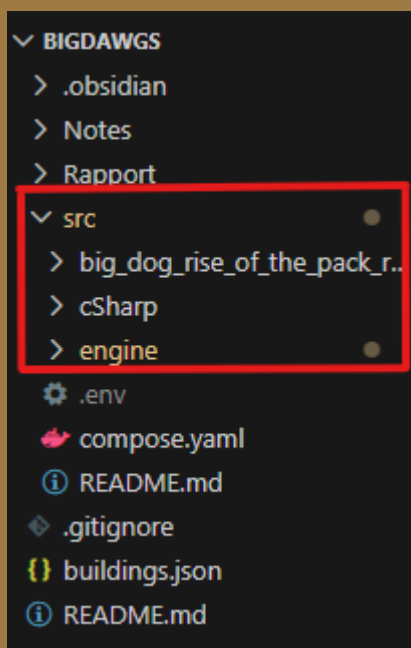
Watch Paths

Enter a path to watch (e.g., src/**, dist/*.js)

Add

Enable Submodules

Bilag 16:



Bilag 17:

Provider

Select the source of your code

 GitHub  GitLab  Bitbucket  Gitea  Git  Raw

Preview Compose

Github Account

Big Dawgs - Rise of the Dawg

Repository

bigdawgs

View Repository

Branch

main

Compose Path

src/compose.yaml

Trigger Type ⓘ

On Push

Watch Paths ⓘ

Enter a path to watch (e.g., src/**, dist/*.js)

Add

☐ Enable Submodules

Save